Verilog HDL로
디지털 회로설계

Dec 17,2024
by SIKWON

목차

0. 디지털회로설계
1. basys3
2. Vivado 설치
3. project 만들기
4. TEST BENCH 코드 만들기

디지털회로설계

- FPGA란 ?
- FPGA의 응용 프로그램은 무엇인가 ?
- FPGA 프로그래밍이란 무엇 인가 ?
- RTL 이란 무엇 인가 ?
- Netlist란 무엇 인가 ?

디지털회로설계

- **FPGA란 ?**

  FPGA는 "Field Programmable Gate Array"의 약자.
  FPGA는 기본적으로 언제 어디서나 프로그래밍하고 재구성할 수 있는 거대한 게이트 배열.
  FPGA는 실제로 단순한 게이트 배열보다 훨씬 더 복잡.
  일부 FPGA에는 메모리 컨트롤러, 고속 통신 인터페이스, PCIe 엔드포인트 등과 같은 내장 하드 블록이 있다.
  요점은 FPGA 내부에는 원하는 회로를 만들기 위해 임의로 함께 연결할 수 있는 많은 게이트가 있다
  FPGA는 Xilinx, Altera, Microsemi 등과 같은 회사에서 제조함.

디지털회로설계

FPGA의 응용 프로그램은 무엇인가 ?

-구조적으로 FPGA는 본질적으로 상상할 수 있는 거의 모든 디지털 회로를 구축하도록 재구성할 수 있는 게이트의 바다이다.
- 다양한 설계로 장치를 재구성할 수 있는 기능과 함께 이러한 뛰어난 유연성으로 인해 FPGA는 많은 응용 분야에서 ASIC(주문형 반도체)에 비해 더 나은 선택이 된다.
-예를 들어, 딥 러닝, AI 또는 어플리케이션 가속 시스템은 최상의 성능을 달성하기 위해 서로 다른 시간에 다른 알고리즘으로 단일 FPGA를 다시 프로그래밍할 수 있다.
-ASIC은 이러한 상황에서 유연하지 않다. 특정 응용 분야에서는 제조되는 개별 장치의 수가 매우 적다. 이러한 애플리케이션을 위한 ASIC를 설계하고 제조하는 데는 엄청나게 많은 비용이 들 수 있다.
-이러한 상황에서 FPGA는 매우 비용 효율적이지만 강력한 솔루션을 제공할 수 있다.
다음은 특별한 순서 없이 FPGA의 잠재적인 응용 프로그램 중 일부이다.

암호화
ASIC 프로토타이핑
산업, 의료 및 과학 기기
오디오/비디오 및 이미지 처리 및 방송
고성능 컴퓨팅, AI 및 딥 러닝
군사 및 우주 응용 분야
네트워킹, 패킷 처리 및 기타 통신.

디지털회로설계

FPGA 프로그래밍이란 무엇입니까?

FPGA 프로그래밍 또는 FPGA 개발 프로세스는 FPGA에서 솔루션을 계획, 설계 및 구현하는 프로세스이다.
계획의 양과 유형은 응용 프로그램마다 다르다. 그러나 모든 특정 요구 사항을 캡처하는 요구 사항 문서를 만들고 제안된 솔루션을 구현하는 방법을 설명하는 디자인 문서를 만들면 잠재적인 문제를 열거하고 관련 계획을 세우는 데 매우 도움이 될 수 있다. 양질의 디자인 문서를 만드는 데 약간의 시간을 투자하면 나중에 리팩토링, 디버깅 및 버그 수정에 많은 시간을 절약할 수 있다. FPGA에서 솔루션을 구현하려면 회로도 또는 Verilog 또는 VHDL과 같은 HDL 코드와 같은 설계 입력 방법 중 하나를 사용하여 설계를 구축하고, FPGA가 이해할 수 있는 출력 파일에 설계를 합성(합성, 넷리스트 생성, 배치 및 라우팅 등)하고, 프로그래밍 도구를 사용하여 출력 파일을 물리적 FPGA 장치에 프로그래밍하는 것이 포함된다.
회로도를 사용하여 설계를 입력하는 것은 더 이상 업계에서 널리 사용되지 않는다.

RTL이란 무엇입니까?

RTL은 Register Transfer Level의 약자 Register Transfer Logic 또는 Register Transfer Language라는 용어도 접할 수 있으며, 이는 모두 하드웨어 설계의 맥락에서 동일한 의미를 갖는다.
RTL은 디지털 하드웨어 설계를 할때 사용하는 추상화 수준중 하나로 하드웨어의 동작을 레지스터간 데이터의 흐름으로 모델링 하는 방식이다.
게이트 모델링은 기본 게이트를 사용하여 하드웨어를 설명하는 것을 의미.
RTL은 소프트웨어 프로그래밍에 사용되는 "의사 코드"라는 용어와 유사한 것으로 생각할 수도 있다. 하드웨어 설계는 각 clock cycle에서 한 세트의 registers에서 다음으로 이어지는 데이터의 단계(또는 흐름) 시퀀스로 설명할 수 있다.
따라서 RTL은 일반적으로 디지털 회로설계, "데이터 흐름" 디자인이라고도 한다.

디지털회로설계

RTL 설계의 특징

RTL 회로설계는 디지털 회로를 다음과 같은 관점에서 기술하는 방법이다:

- **레지스터(Register):** 데이터를 저장하는 하드웨어 요소
- **전이(Transfer):** 클럭 신호에 따라 레지스터 간 데이터가 어떻게 이동하는지
- **논리 연산(Logics):** 데이터가 이동하는 과정에서 어떤 조작이 가해지는지 (예: 가산기, 멀티플렉서 등)

디지털회로설계

- 1990년대 하드웨어 설계자
  - 부울 대수, 카노맵 등을 이용하여 회로 설계
  - 논리 게이트를 이용하여 회로도(schematic)를 그려 회로 설계
  - 디지털 시스템이 복잡해져서 더 이상 회로도 기반으로 설계하기 어려움
  - 이미 설계한 회로를 재사용 하는데도 어려움이 있음

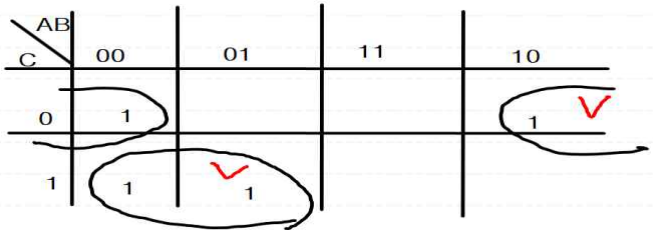
- 인텔 Core2Duo
  - 약 3억 개의 트랜지스터로 설계됨
    (300 million transistors)
  - 2입력 NAND게이트로 환산하면
    약 7천5백만 게이트
  - 기존 방법으로는 설계 불가능
- 최고 제원인 코어 i9-13900K / 14900K의 경우, 칩 면적과 인텔 7 공정의 집적도를 계산했을 때 최대 약 260억 개 수준의 트랜지스터가 내장된 것으로 추정. i7 모델은 이보다 약간 적은 약 200억개 전후
- 하드웨어의 복잡도가 증가하면서 새로운 방식의 설계 방법이 필요하게 됨

디지털회로설계

$X = \bar{A}\,\bar{B}\,\bar{C} + \bar{A}\,\bar{B}\,C + \bar{A}\,B\,C + A\,\bar{B}\,\bar{C}$

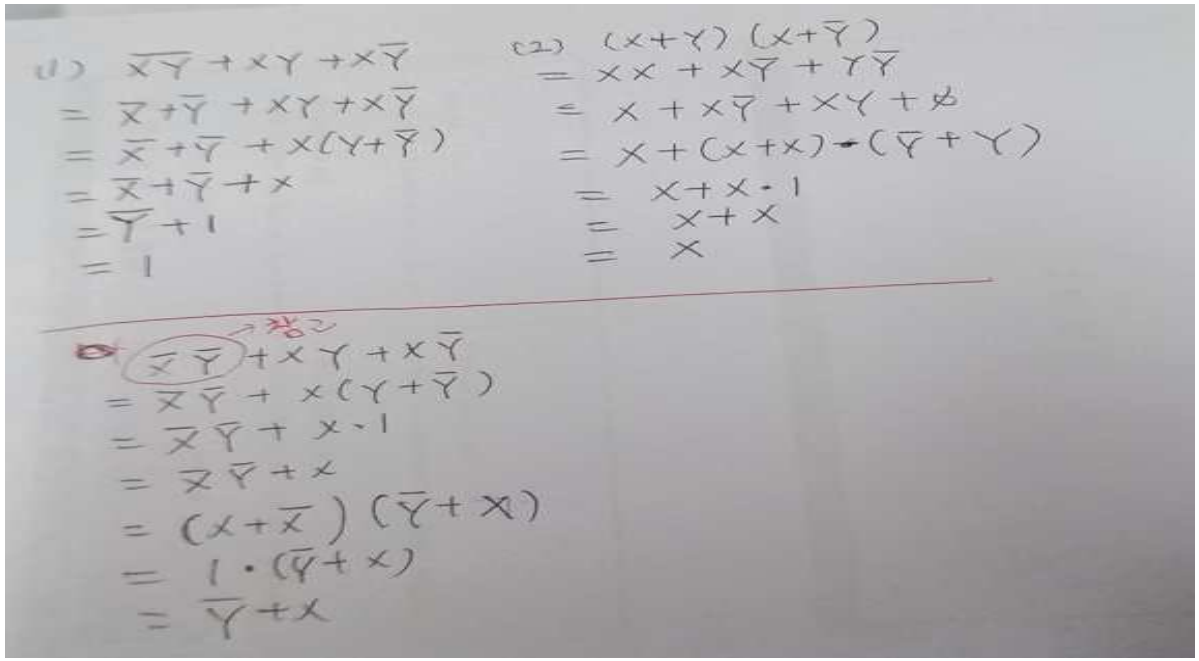
< 진리표 >

A	B	C	X
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	0



$X = \bar{B}\,\bar{C} + \bar{A}C$

<카르노맵 이용 논리식 간소화>



<부울대수 논리식 간소화>

디지털회로설계/HDL(Hardware Description Language)

- HDL:하드웨어의 동작(디지털논리회로)을 기술하는 프로그래밍 언어
  - 컴퓨터가 우리가 설계하고자 하는 회로의 요구사항을 이해할 수 있도록 프로그래밍
  - 하드웨어를 언어로 기술하기 때문에, 회로도로 표현하는 것보다 읽기 편하고, 재사용 하기도 편함

(1) Verilog HDL : 통상 verilog라고 부름
- 1984년 Gateway Design Automation 개발
- IEEE standard (1364), 1995
- 문법이 간결하고 편리하여 산업 현장에서 널리 사용됨

(2) VHDL
- 1981년 Department of Defense 개발
- IEEE standard (1076), 1987

디지털회로설계/HDL(Hardware Description Language)

- Verilog를 배워 얻은 장점은 무엇 인가 ?
  비전공자, 소프트웨어 개발자, 임베디드 개발자
  - 하드웨어 구조 이해
  - 원가 절감
  - 새로운 가능성 추구, 새로운 개발 영역 확장
    (MCU로 구현 못했던것을 가능 하게 한다)
  - 재미

Verilog

- IEEE1364로 표준화된 베릴로그(Verilog)는 전자 회로 및 시스템에 사용되는 하드웨어 기술 언어로, 회로 설계, 검증, 구현 등 여러 용도로 사용할 수 있다.

- 문법은 굉장히 간단하다. C언어와 약간의 영어를 할줄 안다면 너무너무 쉬울 것이라고 생각한다. 하지만 이 모든것들을 회로 위에서 돌린다고 생각하면 느낌이 다르다. C언어와 같은 Programming Language는 Program을 돌리기 위한 것이지만 Verilog는 전자회로를 돌리기 위한 언어이다.

- C 언어와 비슷한 문법을 가져서 사용자들이 쉽게 접근할 수 있도록 만들어졌다. 'if'나 'while'과 같은 제어 구조도 동일하며, 출력 루틴 및 연산자들도 거의 비슷하다.

- 다만 C 언어와 달리, 블록의 시작과 끝을 중괄호 기호를 사용하지 않고, 대신에 begin과 end를 사용하여 구분하고, HDL의 특징인 시간에 대한 개념이 포함되었다는 것 등의 일반적인 프로그램과의 다른 점도 존재한다.
- 구조(structure) : 회로의 연결관계, 블록간 인터페이스
- 동작(behaviour) : 신호의 변화에 따른 회로의 반응, 동작 방식, 입력 신호의 변화에 따라 회로가 어떻게 반응하고 작동해야 하는지 그 '논리적 기능과 규칙'을 기술하는 방식을 의미

- 버전 출시일: 2005.11.9
- 파일 확장자: .v

Verilog 예제

```
module main;
  initial
    begin
      $display("Hello world!");
      $finish;
    end
endmodule
```

```
module toplevel(clock,reset);
  input clock;
  input reset;

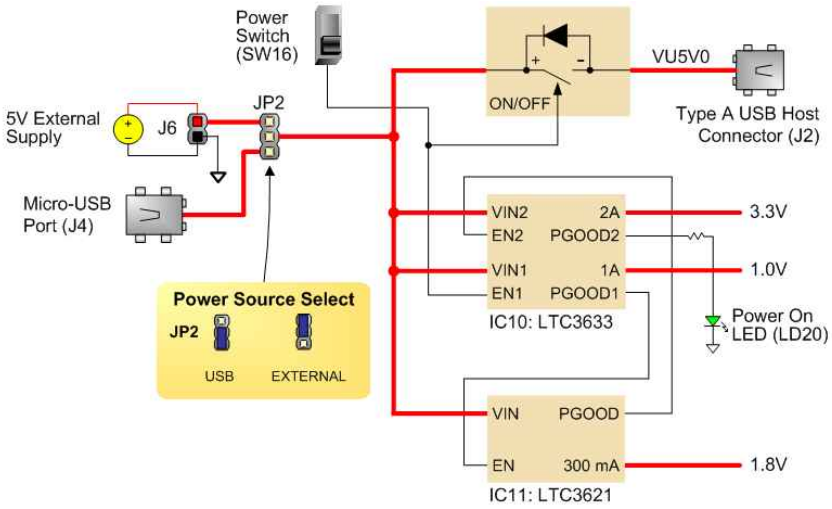
  reg flop1;
  reg flop2;

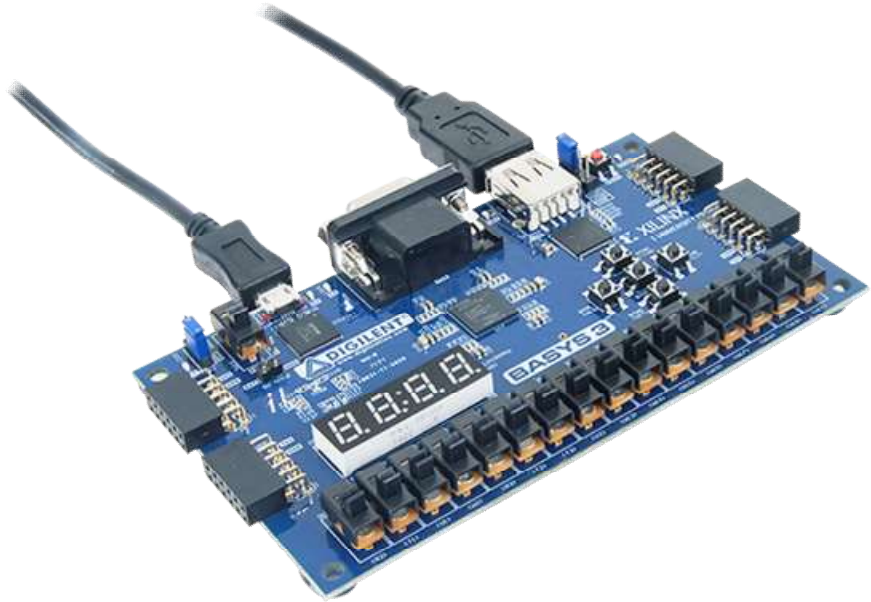
  always @ (posedge reset or posedge clock)
    if (reset)
      begin
        flop1 <= 0;
        flop2 <= 1;
      end
    else
      begin
        flop1 <= flop2;
        flop2 <= flop1;
      end
endmodule
```

베릴로그에서 할당 연산자 "<="는 일반적인 언어와는 다른 기능을 수행한다. 이것은 non-blocking으로 불리며, 클럭(clock)이 발생할 때, 병렬로 동시에 실행된다는 것을 의미한다. 따라서 이 예제에서 flop1과 flop2는 다음의 '클록 주기'(clock cycle) 마다 동시에 서로 바뀌게 될 것이다.
또 다른 할당 연산자 "="는 '블로킹'(blocking)이라 불리며, '클럭'(clock)이 발생할 때, 순차적으로 대입되며 위의 할당이 아래의 할당에 영향을 순차적으로 주게 된다는 것을 의미한다.
만일 위 예에서, "<=" 이 "=" 로 바뀌어 진다면, flop1과 flop2는 서로 변경되지 않고 그 값을 유지하게 될 것이다.

basys3

- basys 3 보드: Xilinx의 Artix-7™ FPGA(Field Programmable Gate Array)를 기반으로 하는
- 디지털 회로 개발 플랫폼
- 조합 회로에서 임베디드 프로세서 및 컨트롤러와 같은 복잡한 순차 회로에 이르기까지 설계 가능
- Artix-7 35T의 특징
  . 5200 슬라이스에 33,280 logic cells (각 슬라이스에는 4개의 6 입력 LUT와 8개의 플립 플롭이 포함됨)
  . 1,800Kbits의 고속 블록 RAM
  . 5개의 클럭 관리 타일, 각각 위상 고정 루프(PLL)가 있음
  . 90 DSP 슬라이스
  . 50MHz를 초과하는 내부 클럭 속도
  . 온칩 아날로그-디지털 컨버터(XADC)





basys3

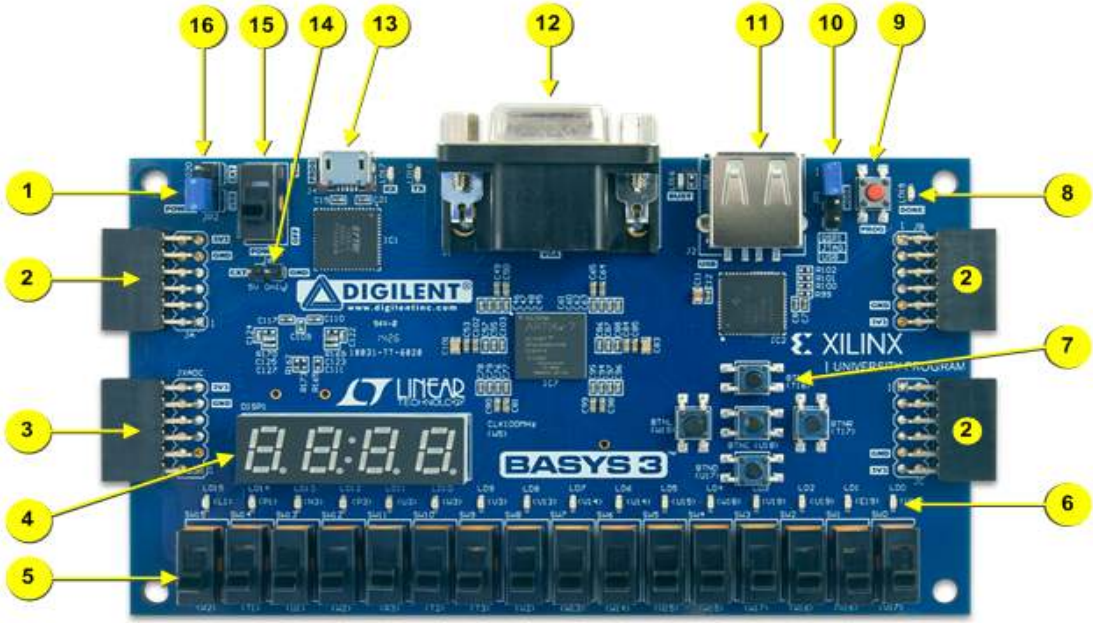


Figure 1. Basys3 board features

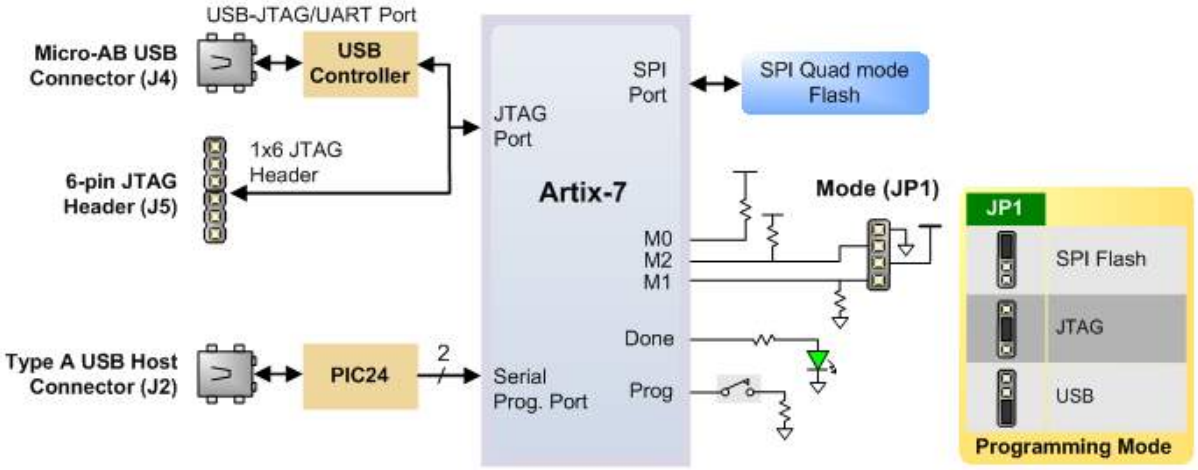
The Basys 3 board includes a single 100MHz oscillator connected to pin W5 (W5 is a MRCC input on bank 34).

Callout	Component Description	Callout	Component Description
1	Power good LED	9	FPGA configuration reset button
2	Pmod connector(s)	10	Programming mode jumper
3	Analog signal Pmod connector (XADC)	11	USB host connector
4	Four digit 7-segment display	12	VGA connector
5	Slide switches (16)	13	Shared UART/ JTAG USB port
6	LEDs (16)	14	External power connector
7	Pushbuttons (5)	15	Power Switch
8	FPGA programming done LED	16	Power Select Jumper

basys3

FPGA 구성(프로그래밍) 3가지 방법

1. PC는 전원이 켜져 있을 때 USB-JTAG 회로(portJ4, "PROG")를 사용하여 FPGA를 프로그래밍할 수 있다.
2. 비휘발성 직렬(SPI) 플래시 장치에 저장된 파일은 SPI 포트를 사용하여 FPGA로 전송할 수 있다.
3. USB HID 포트에 연결된 USB 메모리 스틱에서 프로그래밍 파일을 전송할 수 있다.

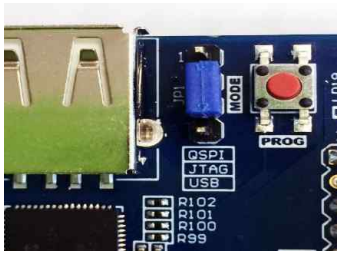


FPGA 구성 데이터는 .bit 파일 확장자를 가진 bitstreams라는 파일에 저장된다.
Xilinx의 Vivado 소프트웨어는 VHDL, Verilog 또는 회로도 기반 소스 파일에서 비트스트림을 생성할 수 있다.
Bitstreams는 FPGA 내의 SRAM 기반 메모리 셀에 저장된다.
이 데이터는 FPGA의 logic functions 및 회로 연결을 정의하며, 보드 전원을 제거하거나,
PROG 입력에 연결된 reset 버튼을 누르거나, JTAG 포트를 사용하여 새 구성 파일을 작성하여 지울 때까지 유효.

basys3

FPGA 구성(프로그래밍) 3가지 방법

1. JTAG 프로그래밍



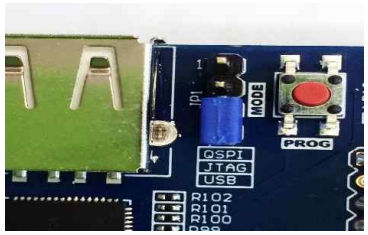
- Xilinx 툴은 일반적으로 JTAG라고 하는 Test Access Port 및 Boundary-Scan Architecture를 사용하여 FPGA와 통신.
-JTAG 프로그래밍 중에는 기판 실장형 Digilent USB-JTAG 회로(포트 J4) 에 연결된 Digilent JTAG-HS2와 같은 외부 JTAG 프로그래머를 사용하여 .bit 파일이 PC에서 FPGA로 전송.
- FPGA가 이미 구성된 경우 JTAG를 통해 전송되는 bitstream로 기존 구성을 덮어쓴다.

2. SPI 프로그래밍



- 비휘발성 플래시 장치를 프로그래밍할 때 비트스트림 파일은 2단계 프로세스로 플래시로 전송된다. 먼저 FPGA는 플래시 장치를 프로그래밍할 수 있는 회로로 프로그래밍된 다음 FPGA 회로를 통해 데이터를 플래시 장치로 전송한다. 플래시 장치를 프로그래밍한 후 모드 점퍼 설정에 따라 결정된 대로 후속 전원 켜기 또는 재설정 이벤트에서 FPGA를 자동으로 구성할 수 있다. 플래시 장치에 저장된 프로그래밍 파일은 전원 주기 이벤트에 관계없이 덮어쓸 때까지 남아 있다.

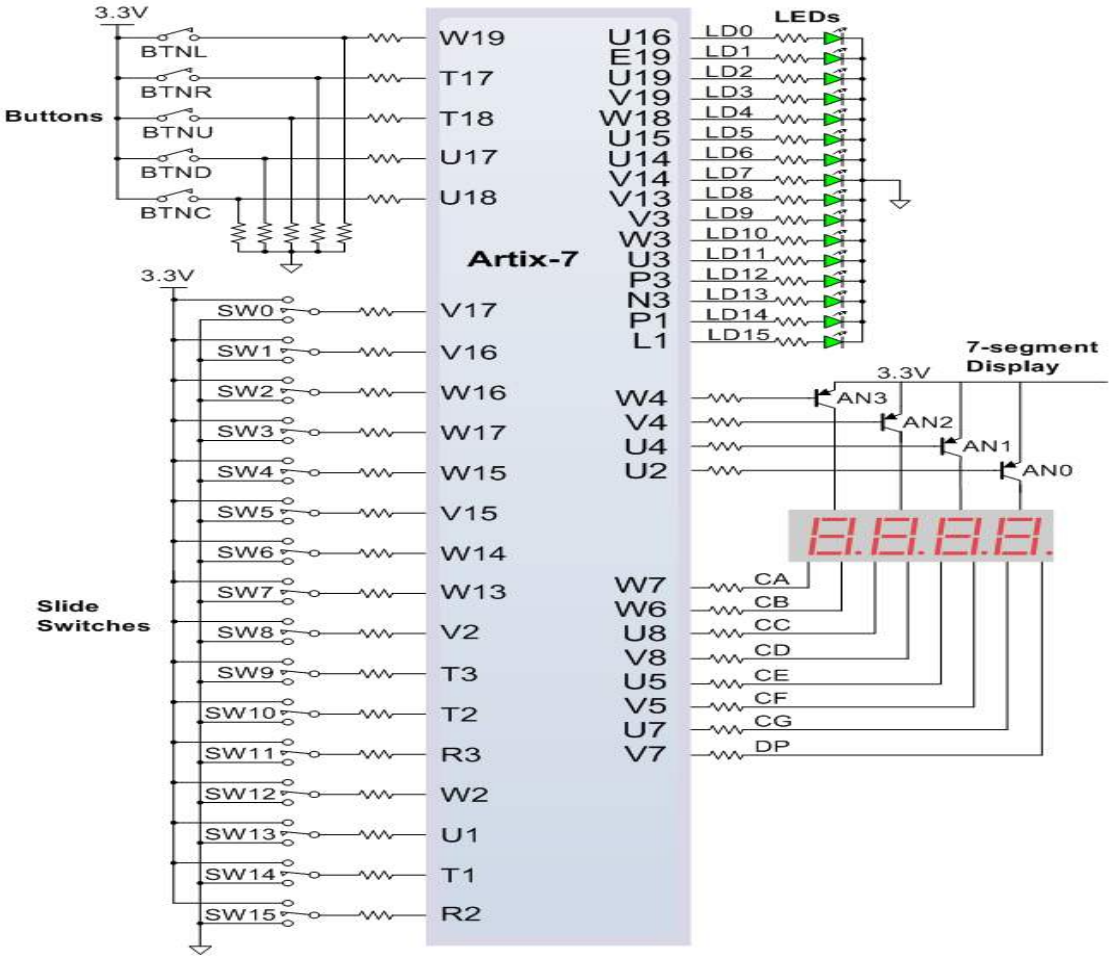
3. USB HOST 프로그래밍

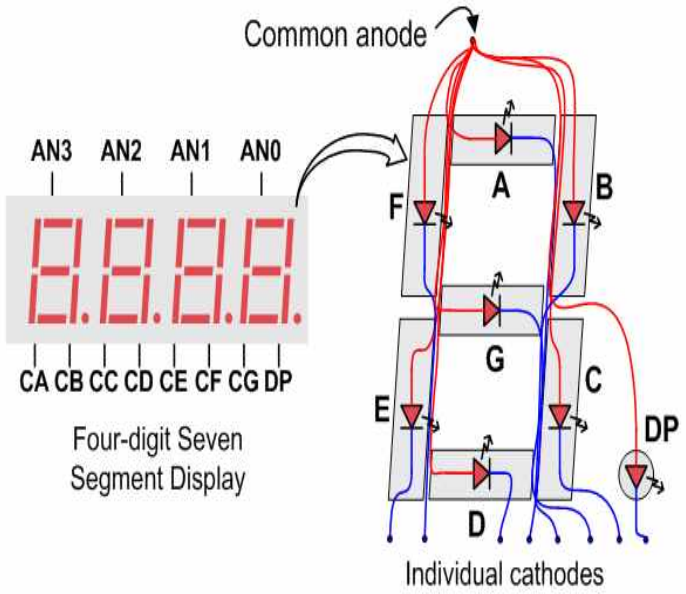


다음을 수행하여 USB-HID 포트(J2)에 연결된 펜 드라이브에서 FPGA를 프로그래밍할 수 있다.
1. 저장 장치(펜 드라이브)를 FAT32 파일 시스템으로 포맷
2. 단일 .bit 구성 파일을 저장 장치의 루트 디렉토리에 배치.
3. 저장 장치를 Basys 3에 연결.
4. Basys 3의 JP1 프로그래밍 모드 점퍼를 "USB"로 설정.
5. PROG 버튼을 누르거나 Basys 3의 전원을 껐다 켠다.

basys3

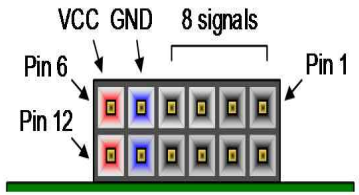
Basys3 I/O



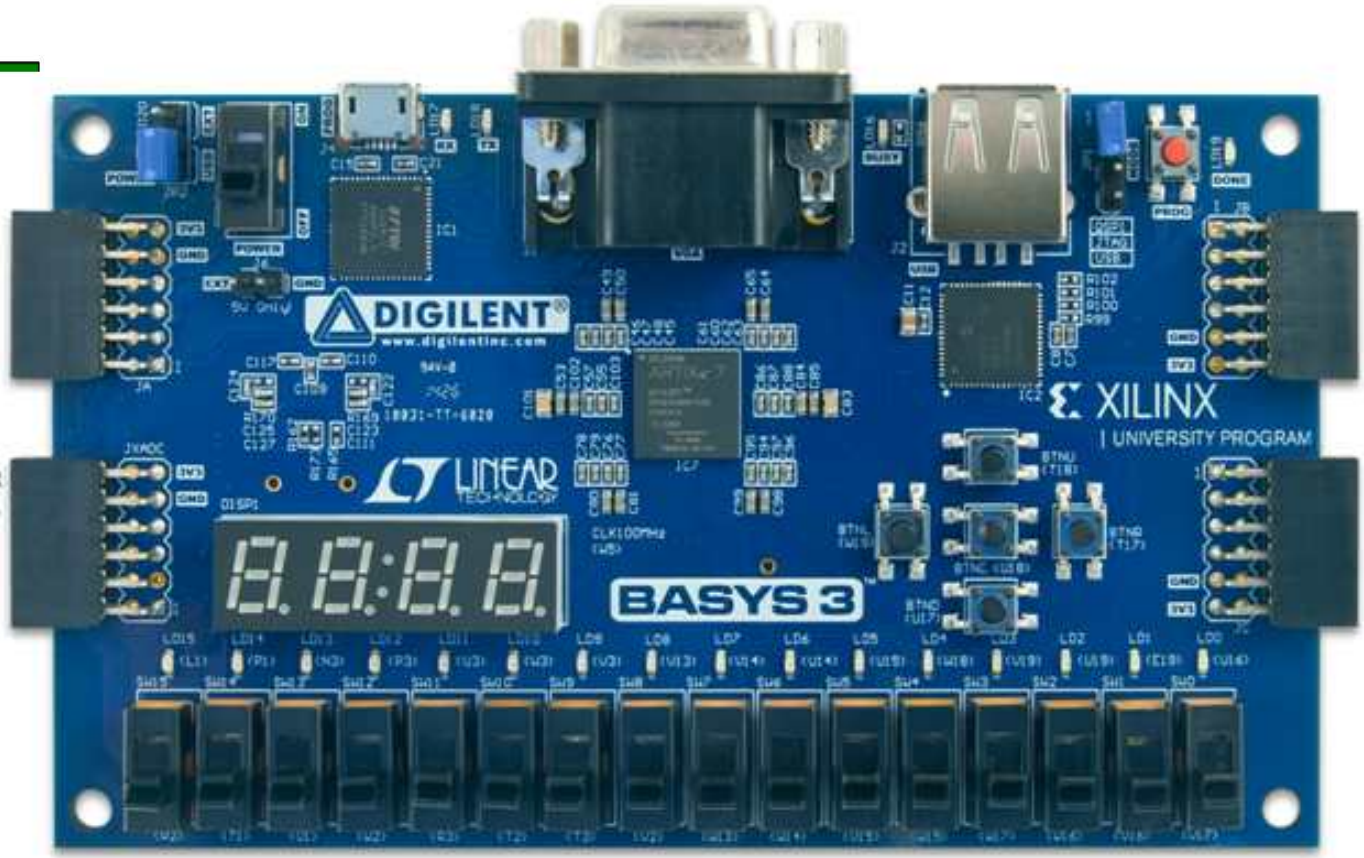


basys3

Pmod 포트



Basys3: Pmod Pin-Out Diagram



JA12 : PWR
JA11 : GND
JA10: G3
JA9: H2
JA8 : K2
JA7: H1

JA6: PWR
JA5: GND
JA4: G2
JA3: J2
JA2: L2
JA1: J1

JXAC12 : PWR
JXAC11 : GND
JXAC10: N1
JXAC9: M1
JXAC8 : M3
JXAC7: K3

JXAC6: PWR
JXAC5: GND
JXAC4: N2
JXAC3: M2
JXAC2: L3
JXAC1: J3

왼쪽

JB1: A14
JB2: A16
JB3: B15
JB4: B16
JB5: GND
JB6: PWR

JB7: A15
JB8: A17
JB9: C15
JB10: C16
JB11: GND
JB12: PWR

JC1: K17
JC2: M18
JC3: N17
JC4: P18
JC5: GND
JC6: PWR

JC7: L17
JC8: M19
JC9: P17
JC10: R18
JC11: GND
JC12: PWR

vivado 설치

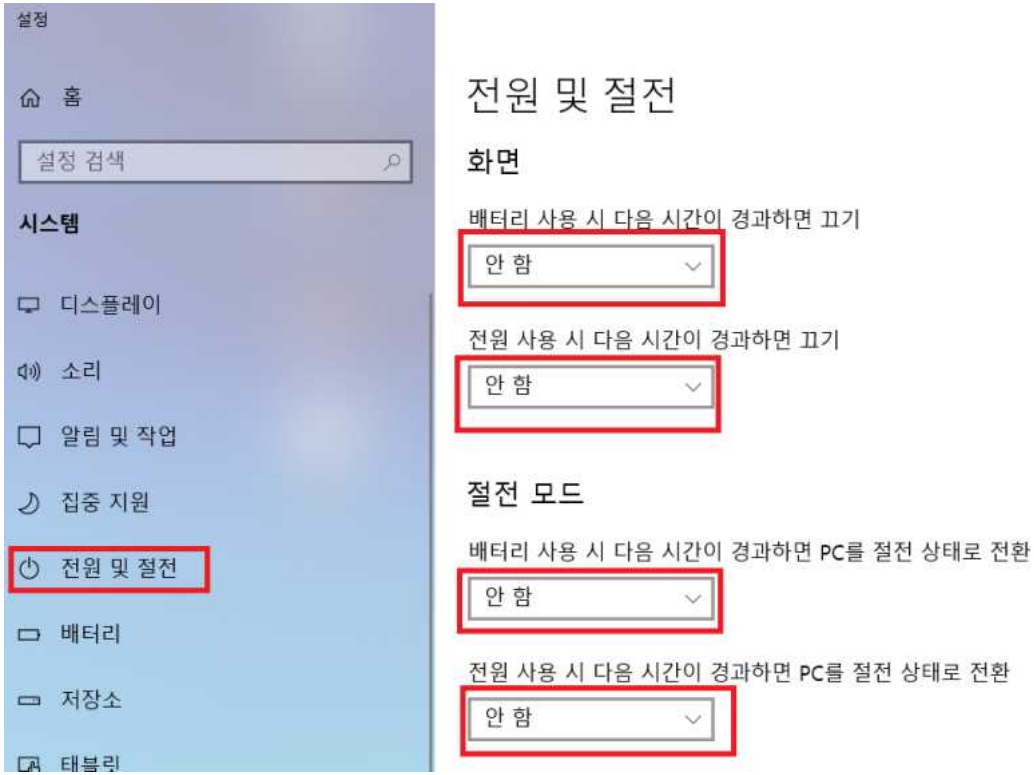
vitis를 설치하면 vivado도 같이 설치 된다.

1. vitis 설치시 PC가 sleep mode로 들어 가지 않도록 pc를 아래와 같이 setup 한다. (삼성 notebook 기준)
sleep mode로 들어 가면 download가 중지 된다.

(1) 바탕화면에서 마우스 우클릭



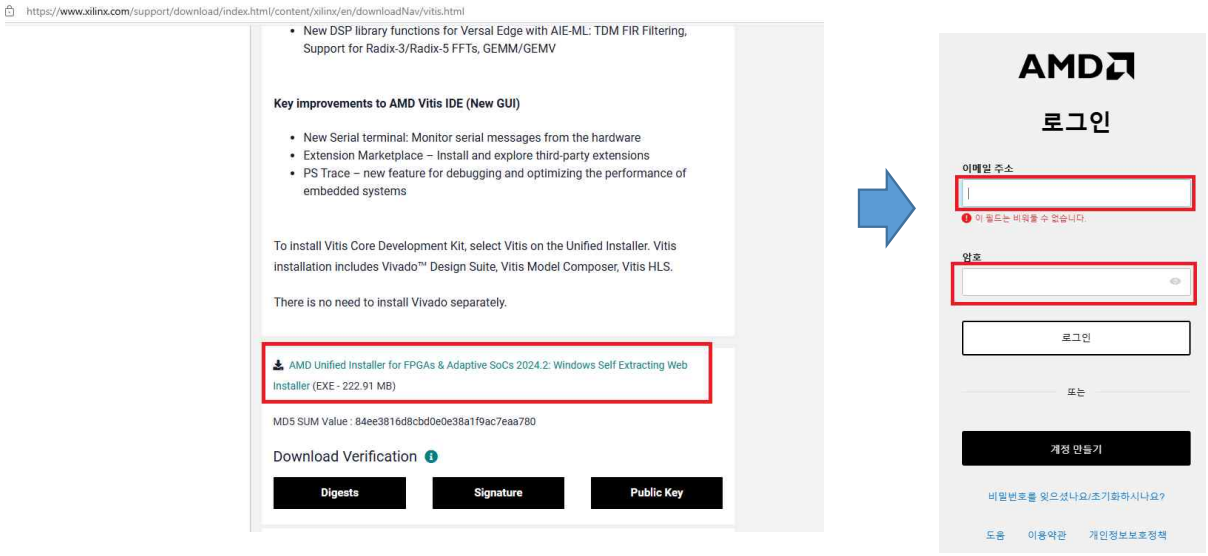
(2) 아래와 같이 전원 및 절전 모드를 사용 안함으로 설정 한다.



vivado 설치

2. www.xilinx.com 홈피에 접속하여 아래와 같이 download를 진행 한다.
   https://www.amd.com/en.html

Products → Software, Tools, & Apps → Vitis Software → Download Now →

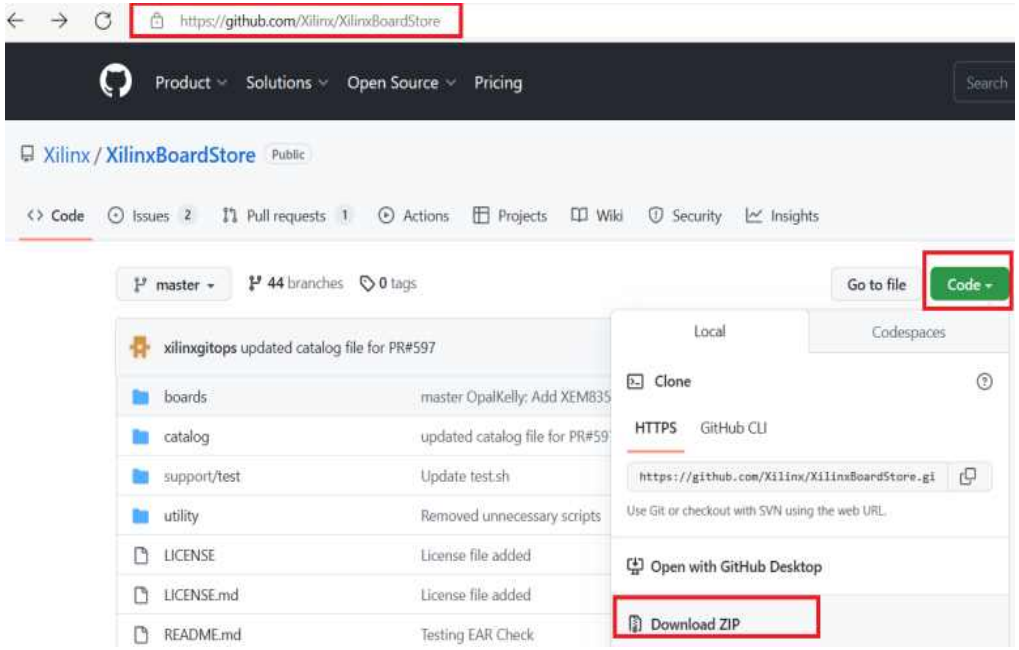


→ Download
→ 설치 한다. (Vitis를 설치 하면 Vivado가 포함 되어 있다)

vivado 설치

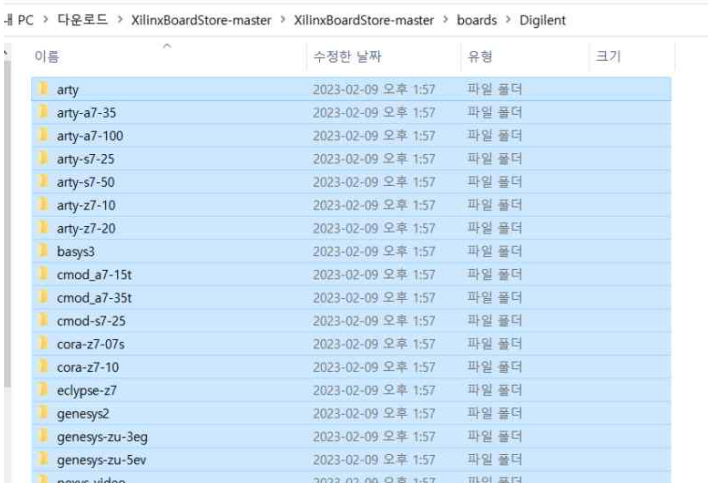
3. Digilent Board 화일 설치
Digilent에서는 각 FPGA 개발 보드에 대한 보드 화일을 제공 한다.
이 화일을 사용하면 새 프로젝트를 만들때 올바른 부품을 쉽게 선택 할 수 있으며 설계 시 여러 복잡한 구성 요소를 자동으로 구성 할 수 있다.
3.1 vivado board git hub에서 download 한다.
GitHub - Xilinx/XilinxBoardStore 를 접속 한다.

3.2 아래의 화일을 Download 한다.



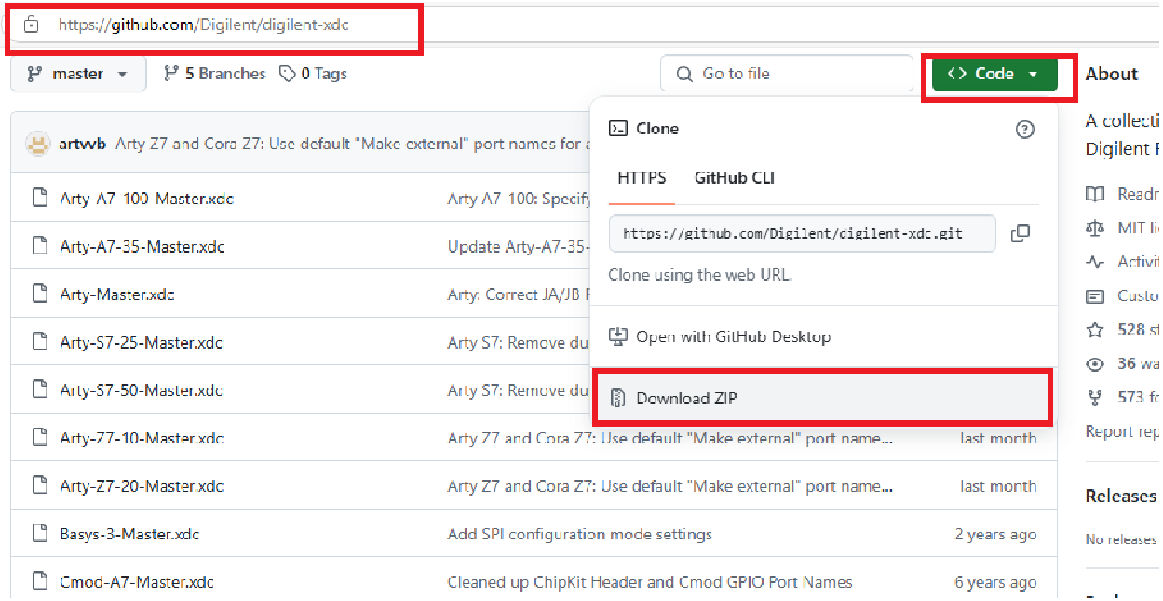
3.3 아래의 Download한 화일을 압축 해제 하여 Digilent 화일을 복사하여 C:₩Xilinx₩Vivado₩2020.2₩data₩boards₩board_files 에 붙여 넣는다

★★ 주의 ★★
Vivado를 처음 설치 하면 **C:₩Xilinx₩Vivado₩2024.2₩data₩boards 디렉토리만 존재한다. board_files 디렉토리를 만들어야 한다.**

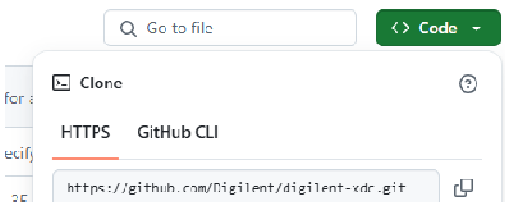


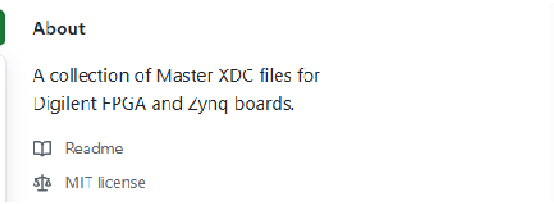
vivado 설치

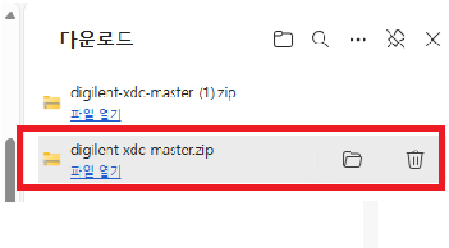
***** Digilent Board xdc 화일 설치 *****
XDC화일을 아래와 같이 다운로드하여 digilent-xdc-master.zip 파일을 압축 해제 하여 사용자 directory 의 적절한 위치에 배치 한다. 예) d:₩project₩FPGA₩doc₩BASYS3₩digilent-xdc-master



※ 위 github에서 다운로드가 잘 안될때는 **digilent-xdc-master.zip를 별도 공급 한다.**



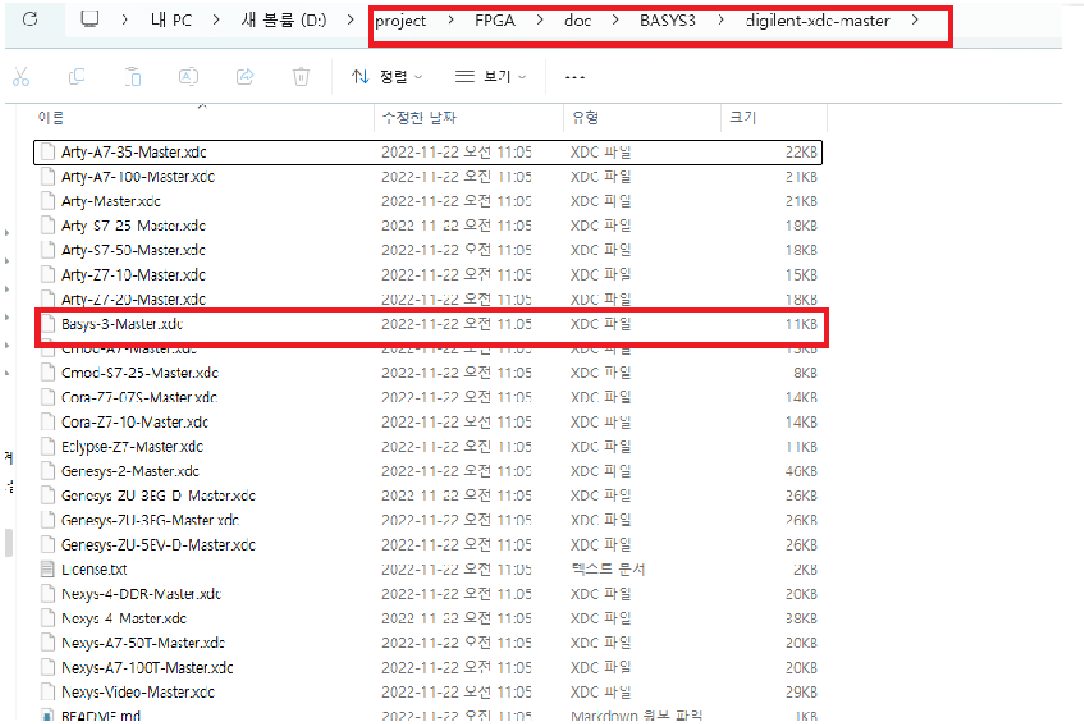




vivado 설치

***** Digilent Board xdc 화일 설치 *****

XDC화일 예) D:₩project₩FPGA₩doc₩BASYS3₩digilent-xdc-master

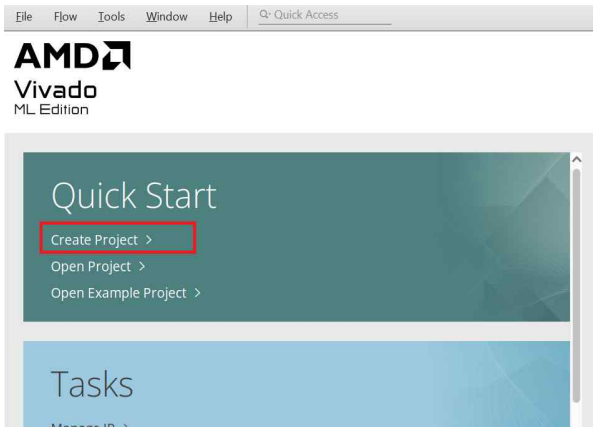


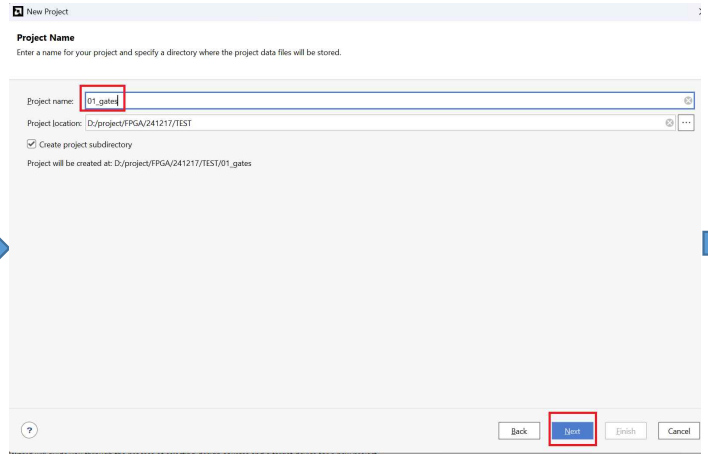
project 만들기

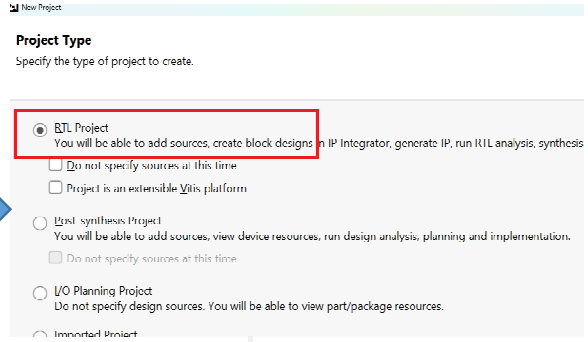
1. vivado를 실행 한다.



2.

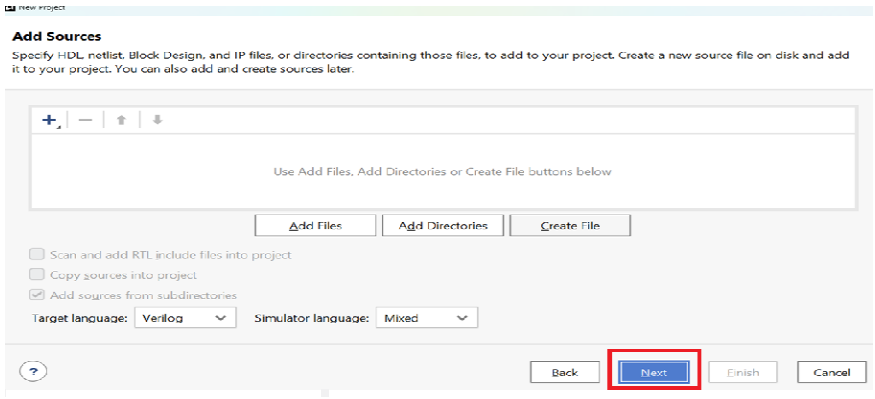






project 만들기

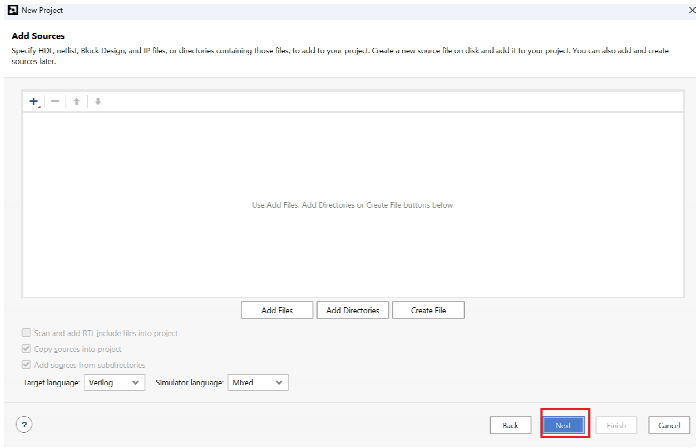
3. source code추가 화면이 나오는데 추가하지 않고 새로 작성 하는 것이므로 next를 누른다

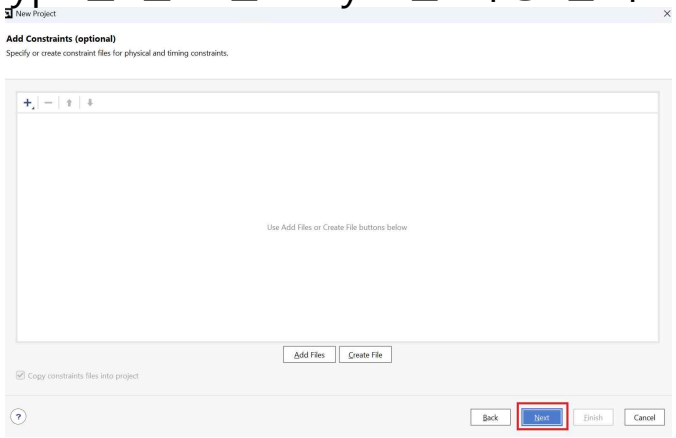


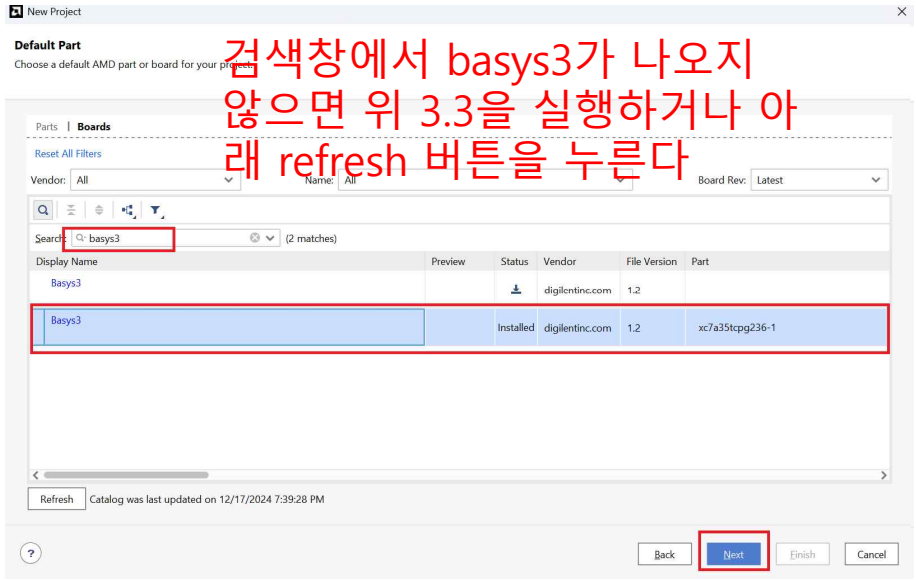
4.Constraints (.XDC 파일) 확인

project 만들기

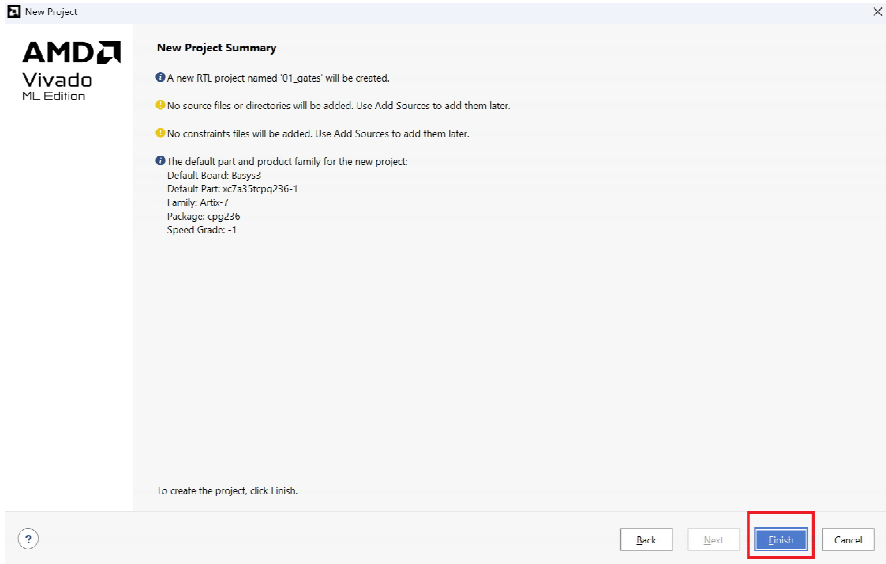
Constraints 을 작성 할 것인지를 물으면 skip하고 board type을 물으면 basys3를 지정 한다.





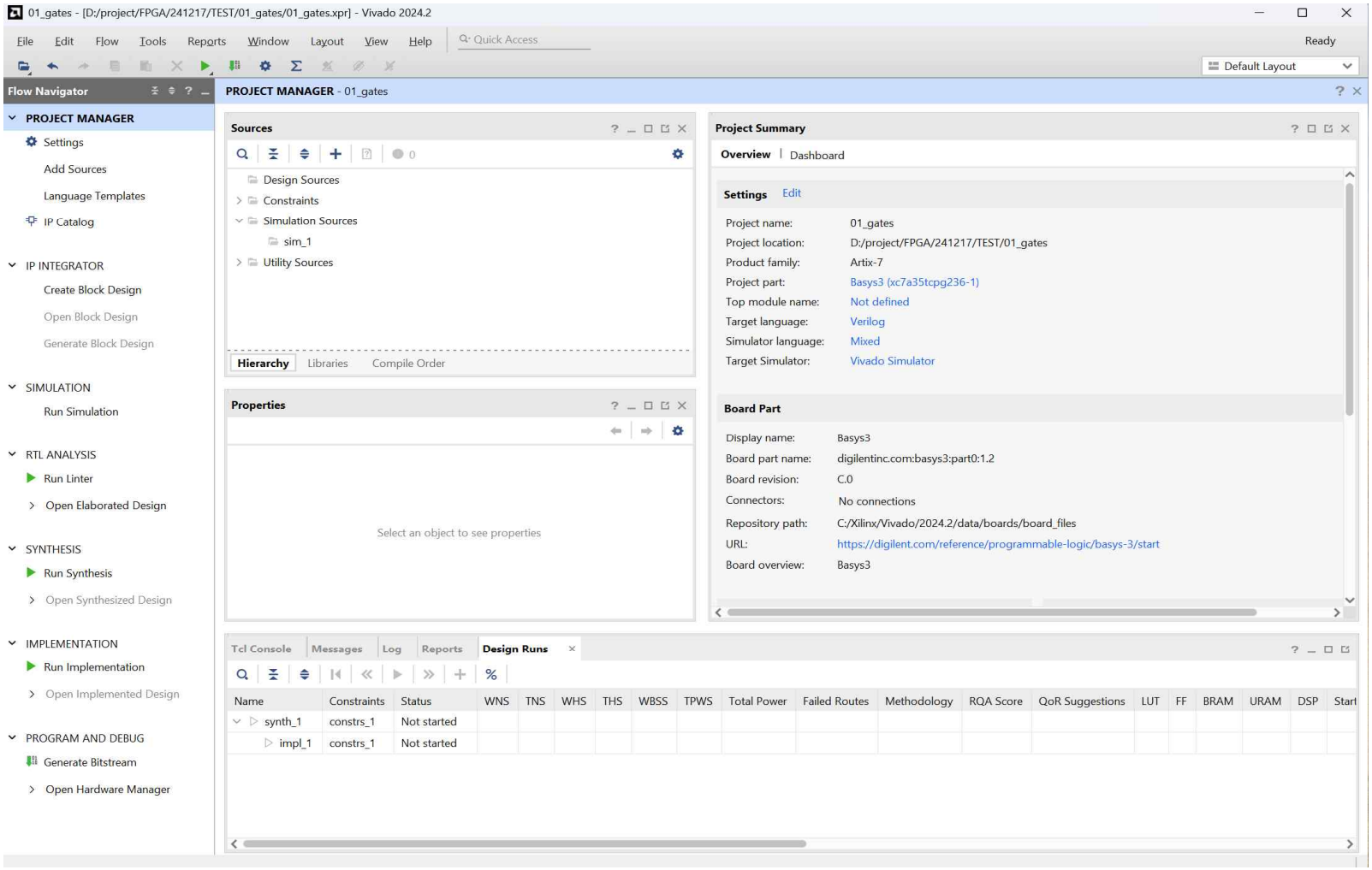


검색창에서 basys3가 나오지 않으면 위 3.3을 실행하거나 아래 refresh 버튼을 누른다



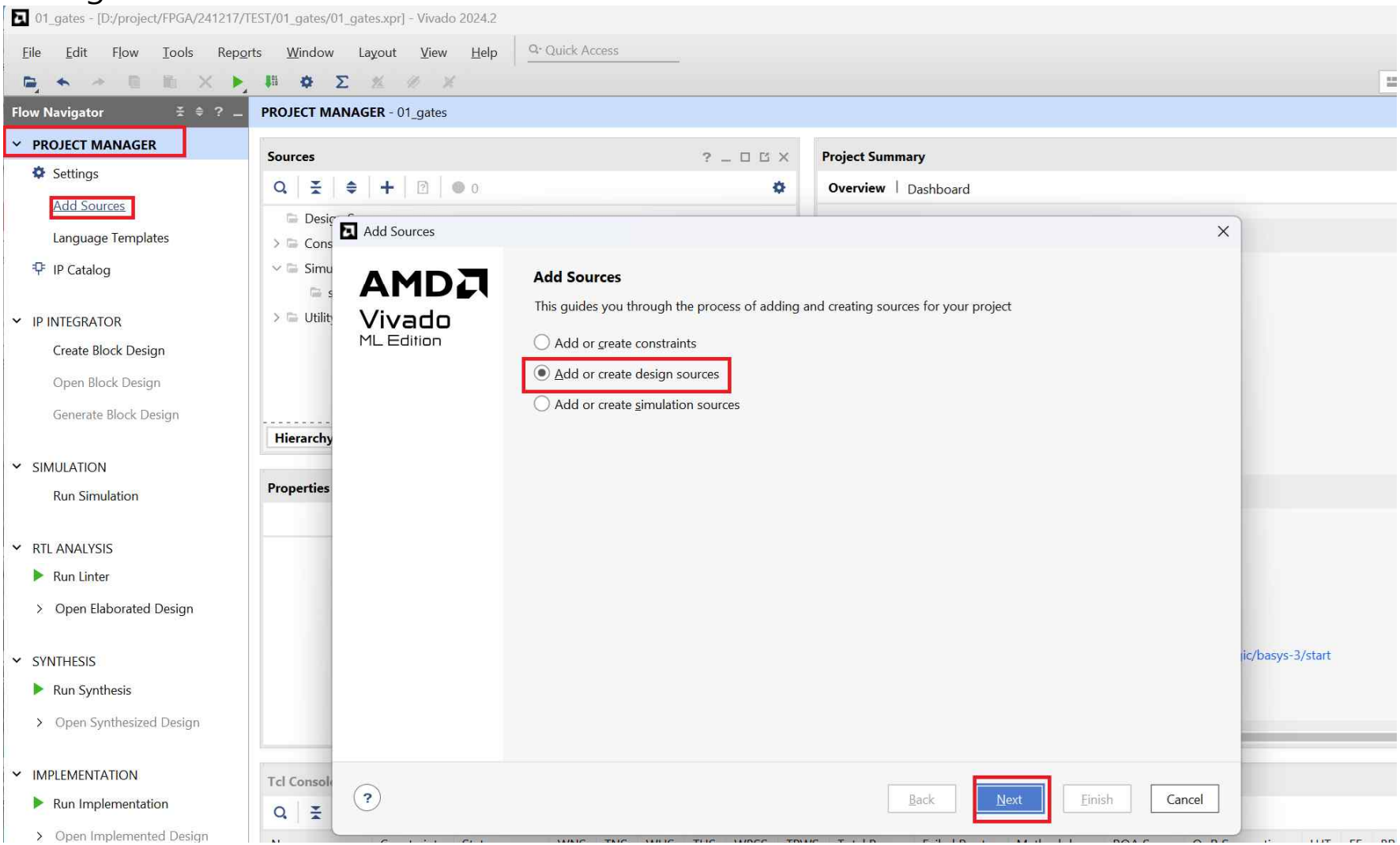
project 만들기

Project 생성을 확인 한다.

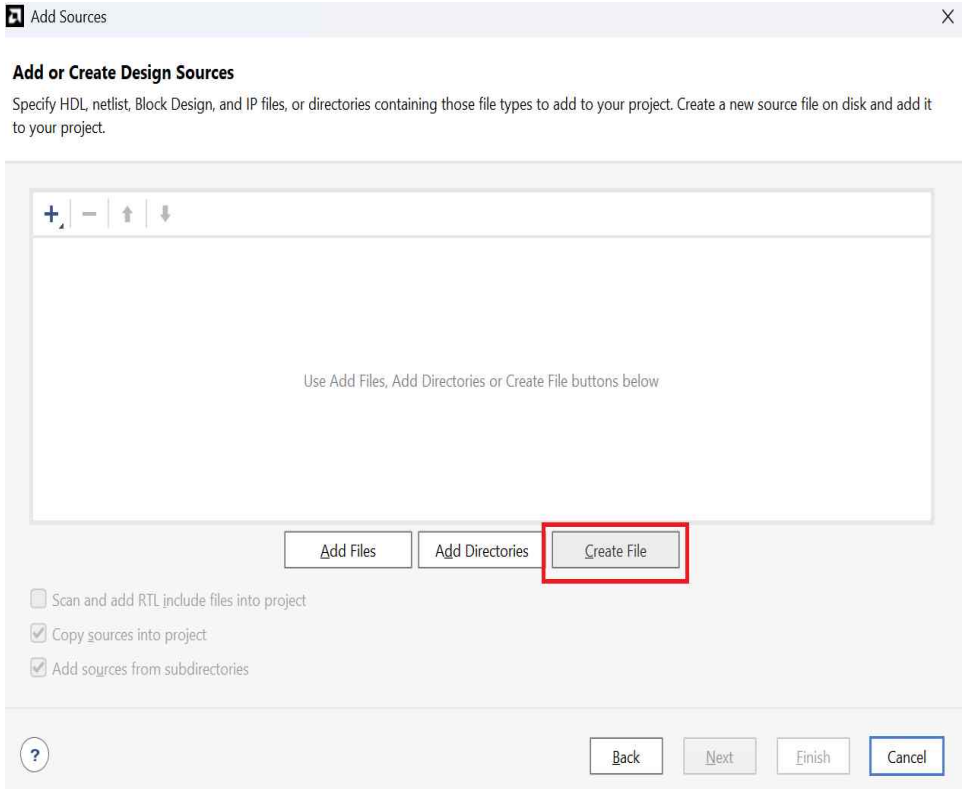


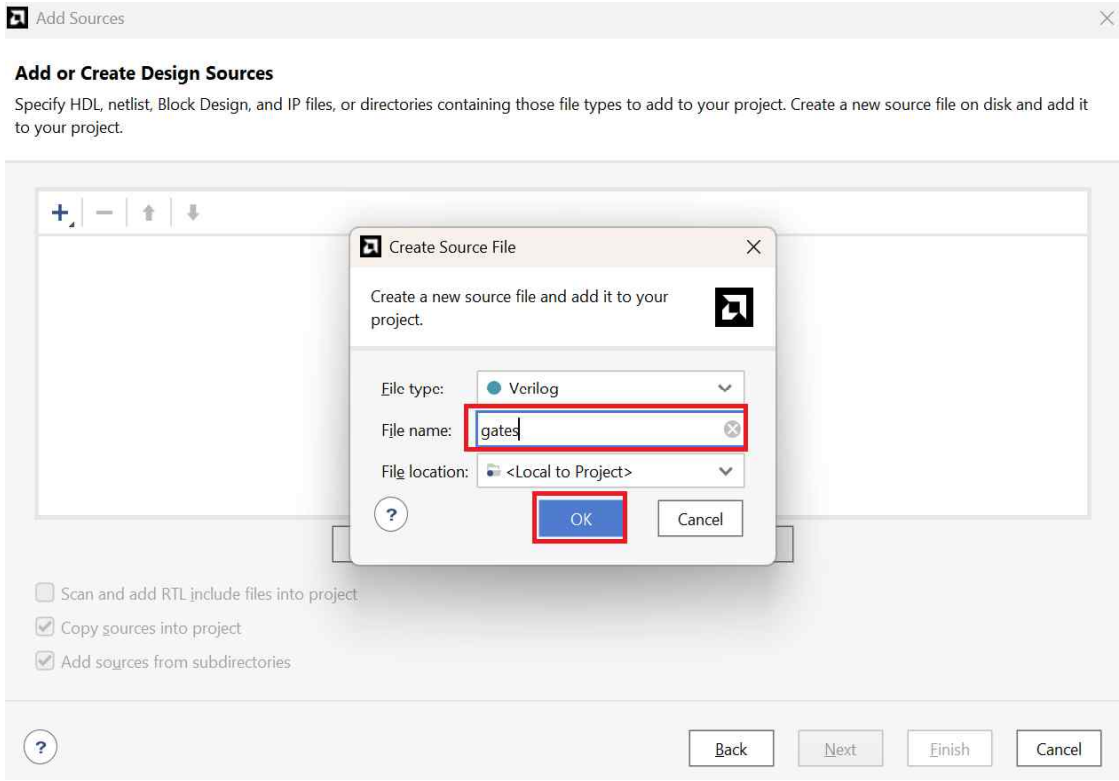
project 만들기

Velilog source code 작성 하기

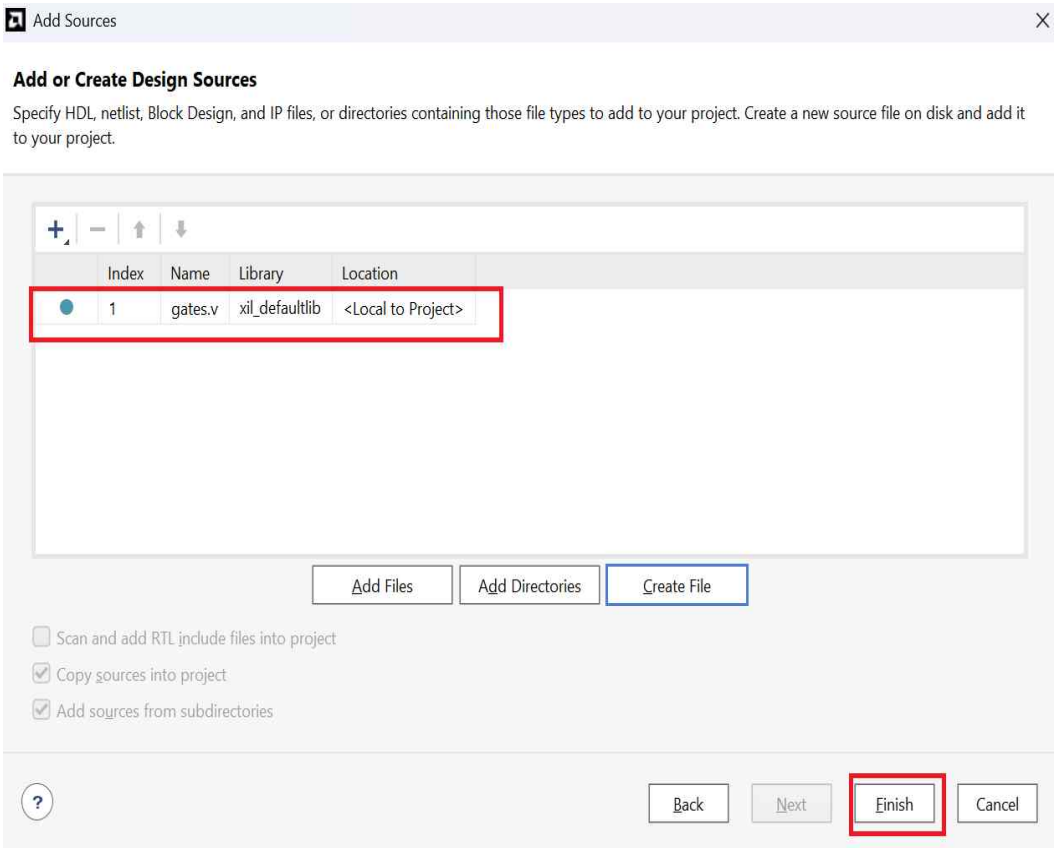


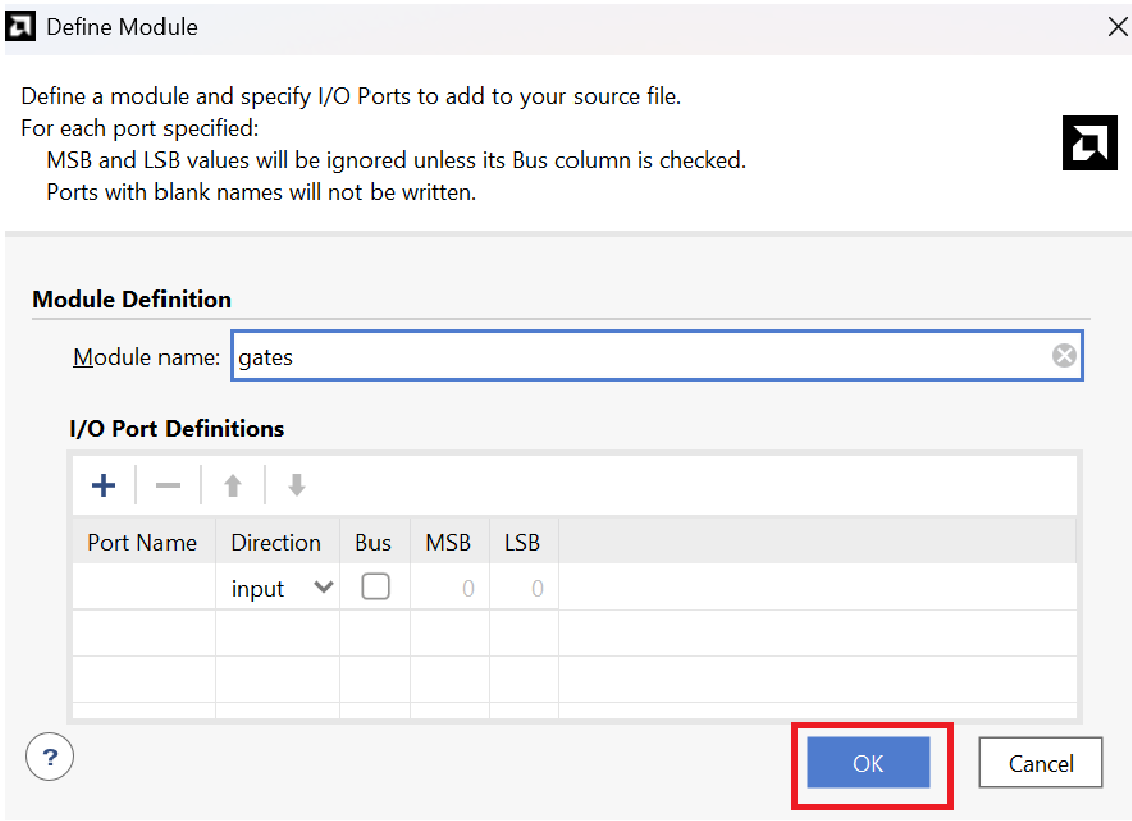
project 만들기


Add Sources
Add or Create Design Sources
Specify HDL, netlist, Block Design, and IP files, or directories containing those file types to add to your project. Create a new source file on disk and add it to your project.
Use Add Files, Add Directories or Create File buttons below
Add Files
Add Directories
Create File
Scan and add RTL include files into project
Copy sources into project
Add sources from subdirectories
Back
Next
Finish
Cancel


Add Sources
Add or Create Design Sources
Specify HDL, netlist, Block Design, and IP files, or directories containing those file types to add to your project. Create a new source file on disk and add it to your project.
Create Source File
Create a new source file and add it to your project.
File type:
Verilog
File name:
gates
File location:
<Local to Project>
OK
Cancel
Scan and add RTL include files into project
Copy sources into project
Add sources from subdirectories
Back
Next
Finish
Cancel

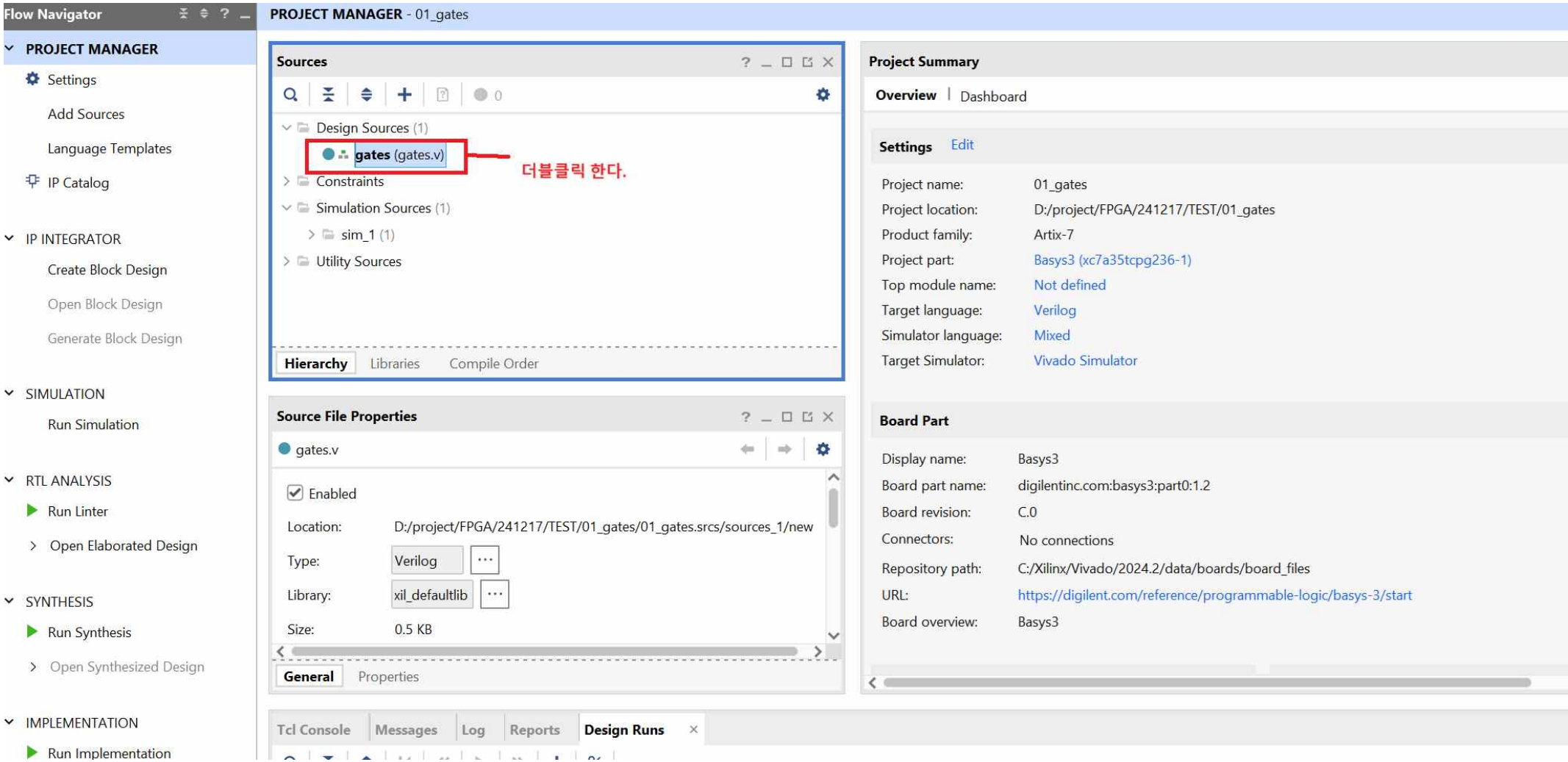
project 만들기


Add Sources
Add or Create Design Sources
Specify HDL, netlist, Block Design, and IP files, or directories containing those file types to add to your project. Create a new source file on disk and add it to your project.
Index
Name
Library
Location
1
gates.v
xil_defaultlib
<Local to Project>
Add Files
Add Directories
Create File
Scan and add RTL include files into project
Copy sources into project
Add sources from subdirectories
Back
Next
Finish
Cancel


Define Module
Define a module and specify I/O Ports to add to your source file.
For each port specified:
MSB and LSB values will be ignored unless its Bus column is checked.
Ports with blank names will not be written.
Module Definition
Module name: gates
I/O Port Definitions
Port Name
Direction
Bus
MSB
LSB
input
0
0
OK
Cancel

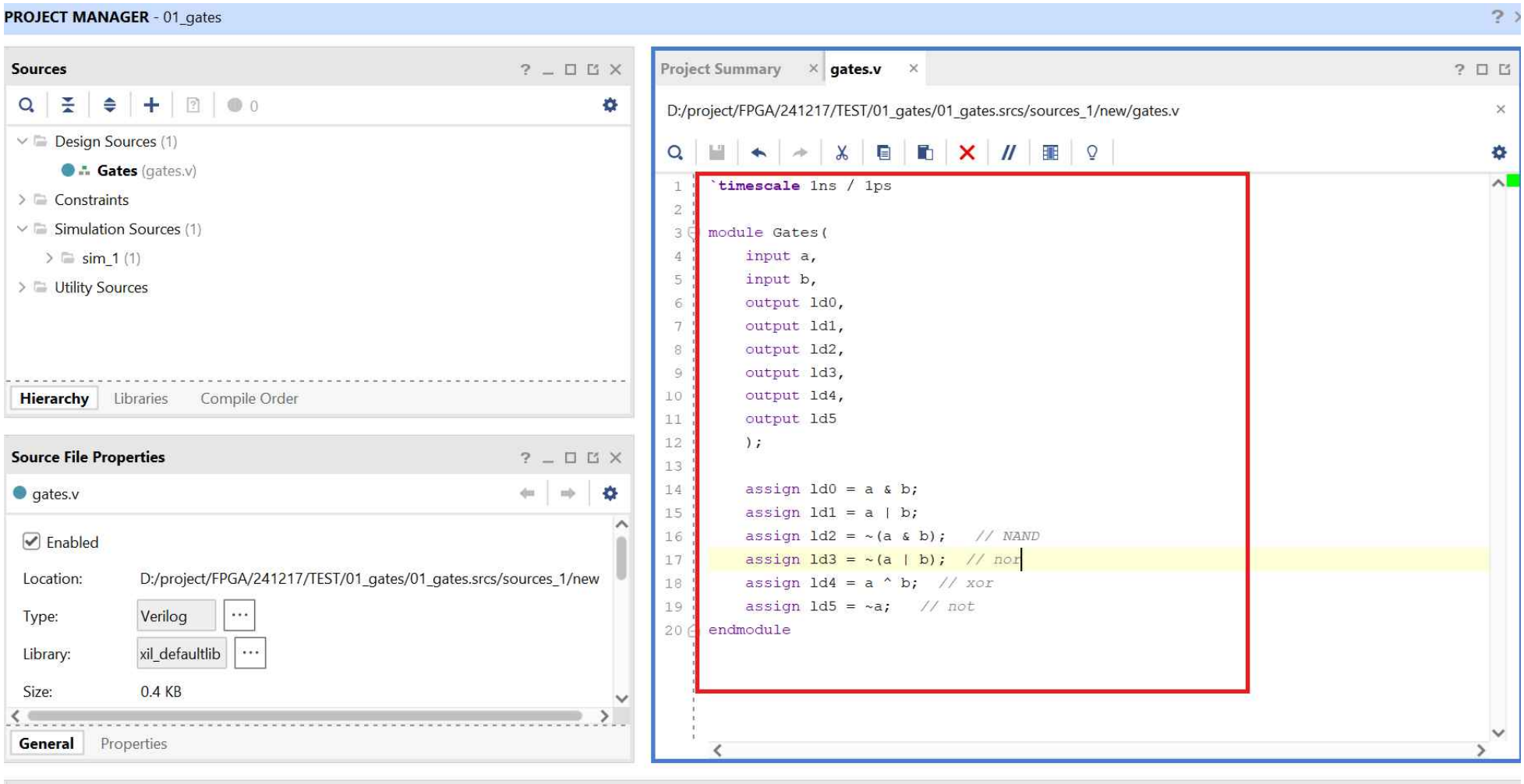
project 만들기

4. source code를 작성 한다.



project 만들기

source code를 작성 한다. assign은 대입이 아닌 연결의 의미를 갖음



project 만들기

module : **논리 회로를 구성하기 위한 기본 단위이다.**
각 module은 입력과 출력 신호를 정의 하고 그 신호들이 어떻게 처리 되는지를 설명하는 logic을 포함한다.

- 키워드: Verilog HDL을 위하여 정의된 것으로 소문자를 사용
  - module, endmodule: 모듈의 시작과 끝
  - input: 입력, output: 출력, inout: 입출력
  - assign, always, initial 등

조합회로: 출력이 현재의 입력에 의해 결정되는 기억 소자가 없는 회로

순차회로: 출력이 현재의 입력과 과거의 입력정보를 포함하여 결정되는 기억 소자

```
module 모듈 이름 (          // 모듈 이름 정의
                  i_a,
                  i_b,      // 모듈의 입출력 포트 이름
                  o_s);
                            // 입출력 포트의 특성을 선언
input i_a;                  // i_a 는 입력 포트임
input i_b;                  // i_b는 입력 포트임
output  [3:0]  o_s;         // o_s는 출력 포트이며, 버스폭은 4비트임
...
assign=....                 // 동작 기술
...
always @(...)               // 동작 기술
...
initial                     // 동작 기술
...
endmodule
```

project 만들기

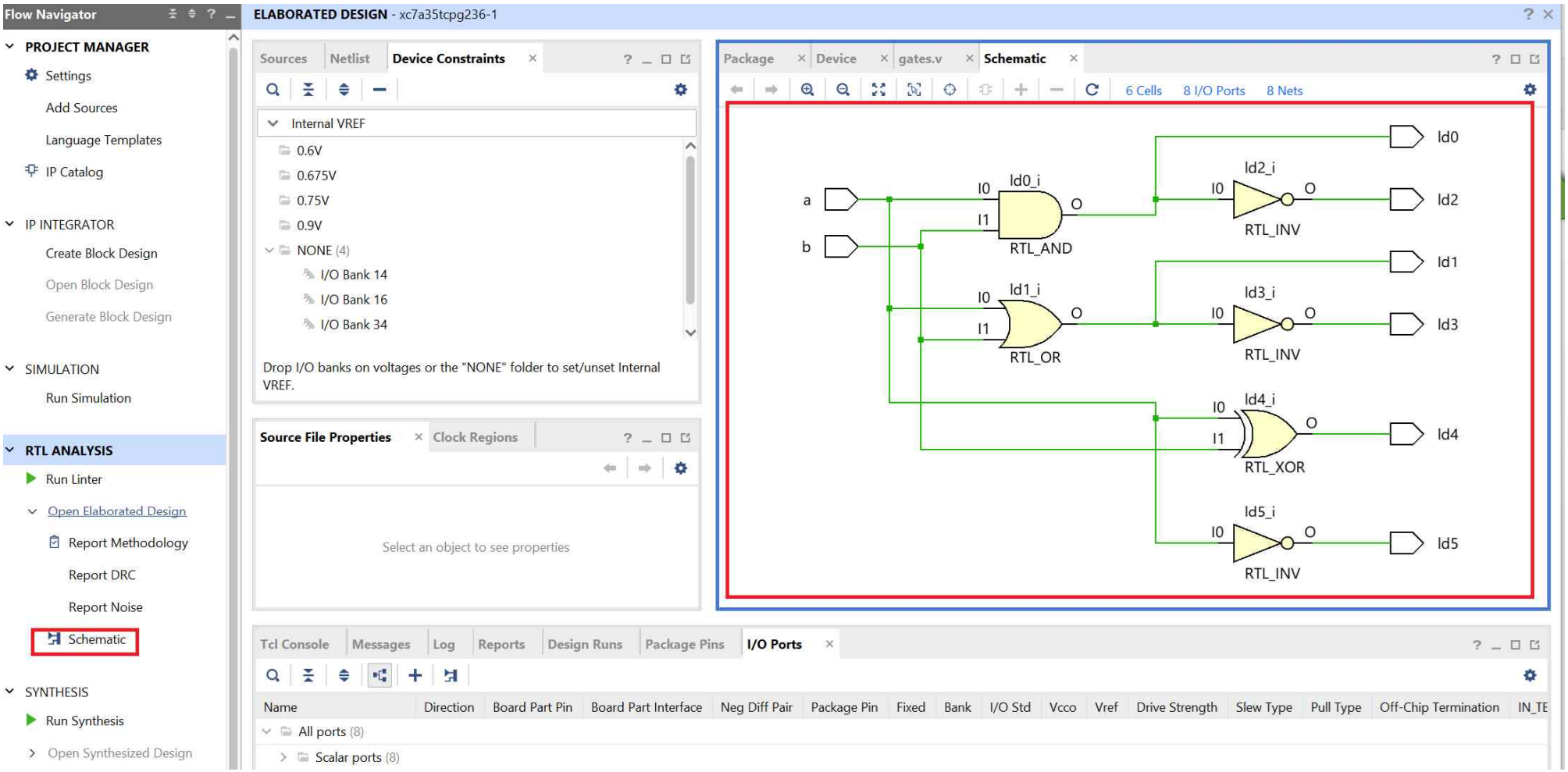
module은 여러 개의 서브모듈로 나눠져 복잡한 시스템을 설계할 수 있도록 도와주는 역할을 한다.

즉, **재사용성**이 뛰어나기 때문에 설계의 효율성을 크게 높일 수 있는 구조이기에 다음과 같은 구조를 사용하는 것이다.

하드웨어의 기능 및 단위를 분리해주기 때문에 중요한 요소이다

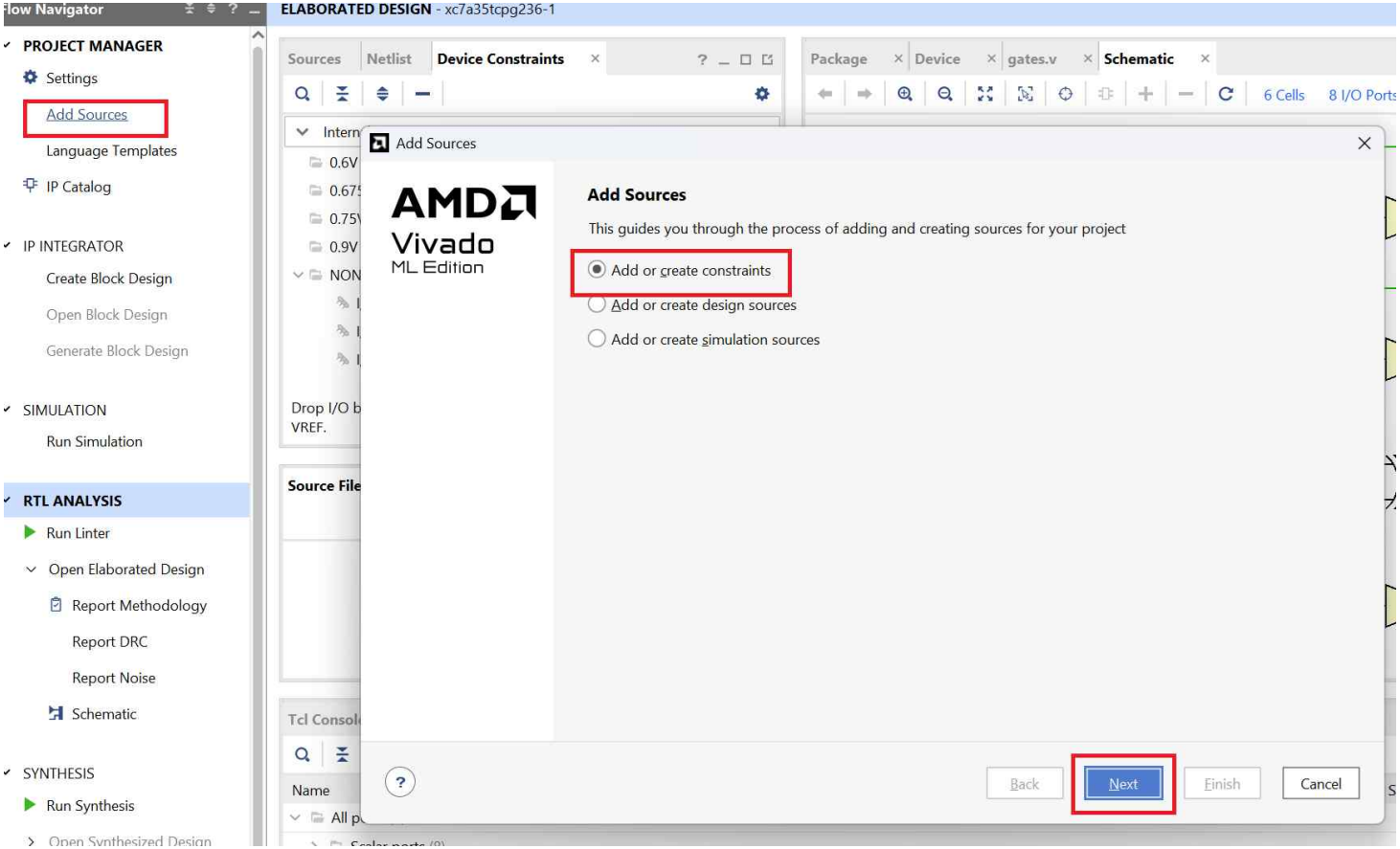
project 만들기

Verilog code가 회로도 로 변환 되었는지 확인



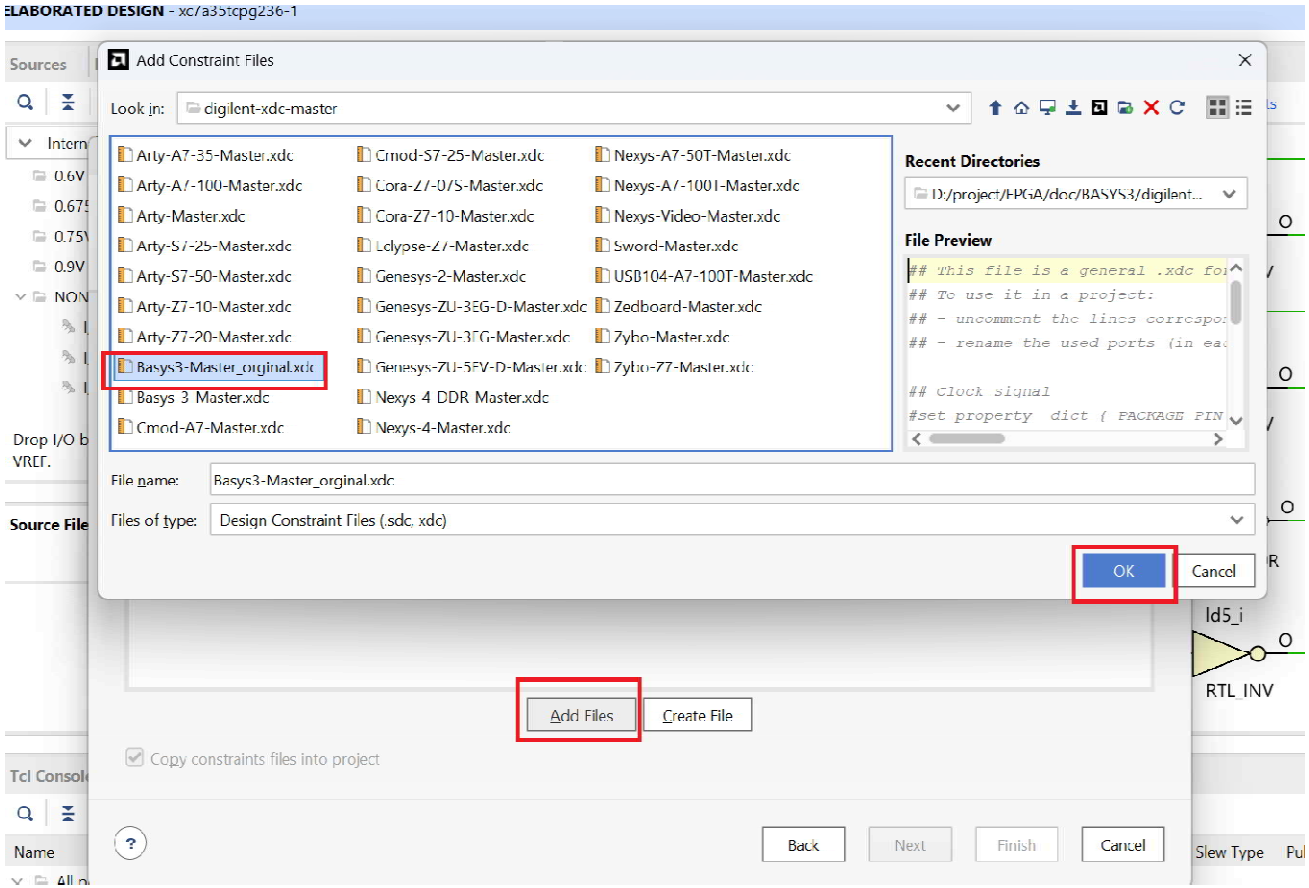
project 만들기

Constraints 파일을 작성 한다.

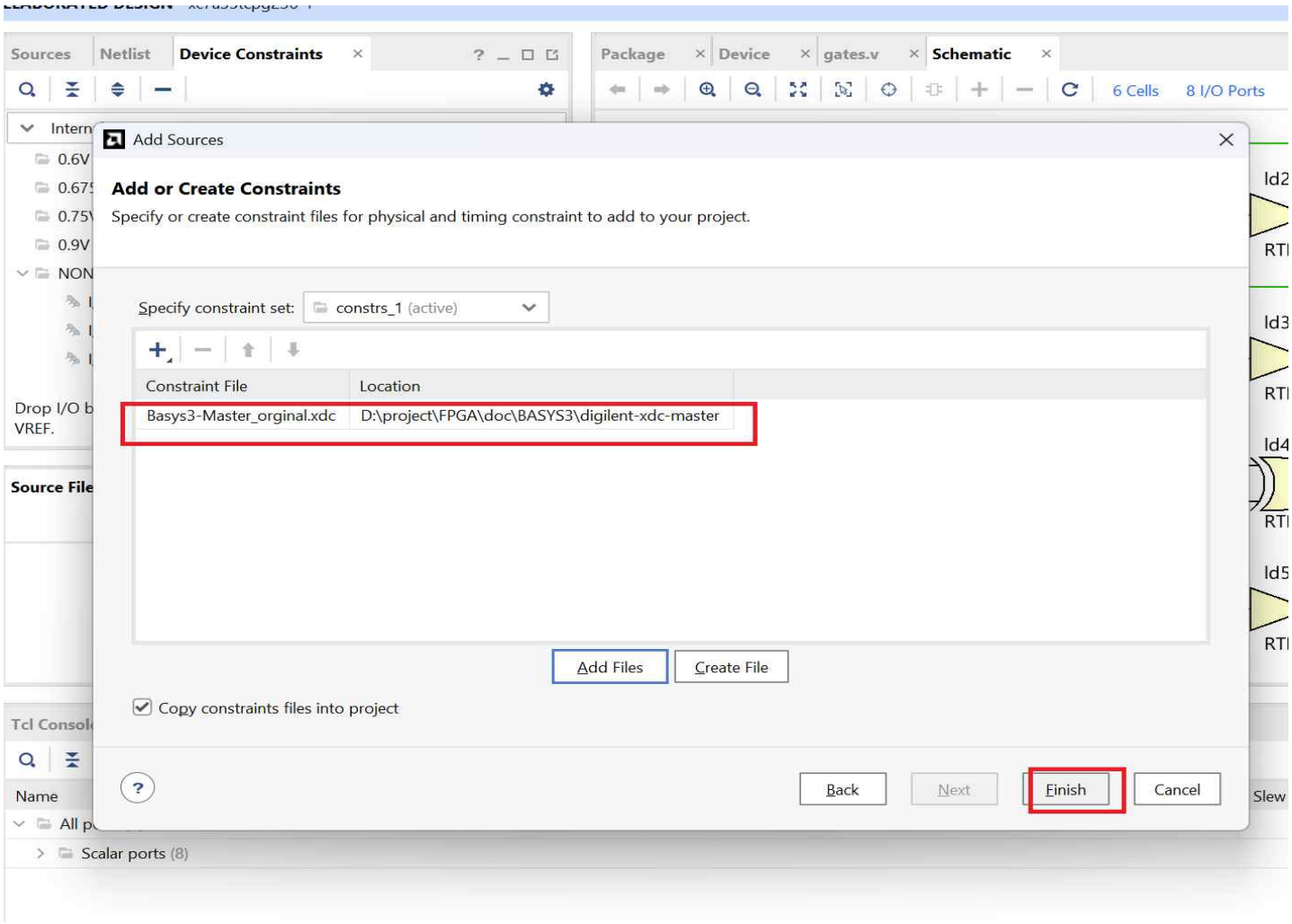


project 만들기

Digilent에서 제공하는 xdc화일을 download한 디렉토리를 지정하여 add한다.
본인은 임의로 **D:₩project₩FPGA₩doc₩BASYS3₩digilent-xdc-master에 download하여 놓았음**
Basys-3-Master.xdc의 원본 파일을 Basys-3-Master_orginal.xdc 파일로 복사 하였다.

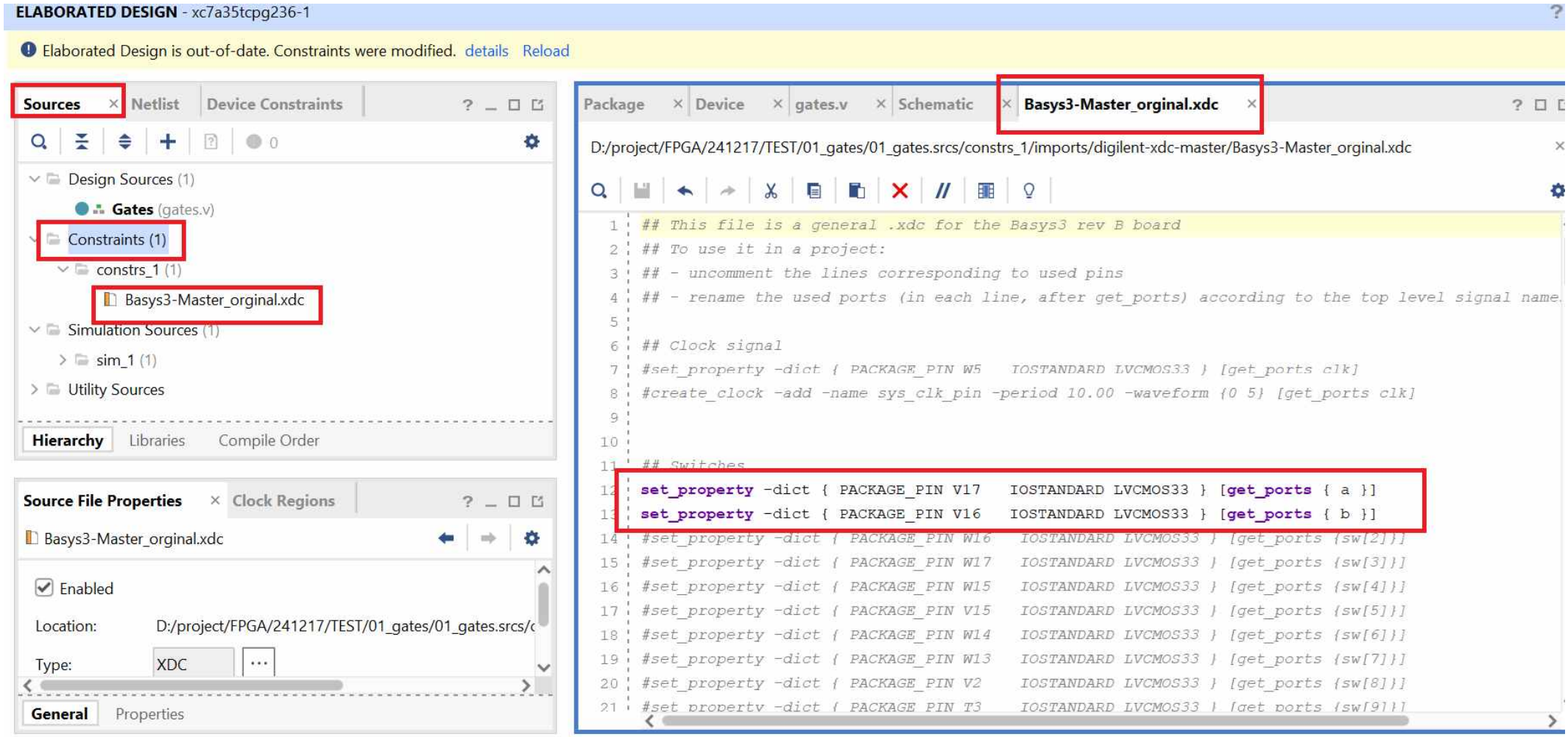


project 만들기

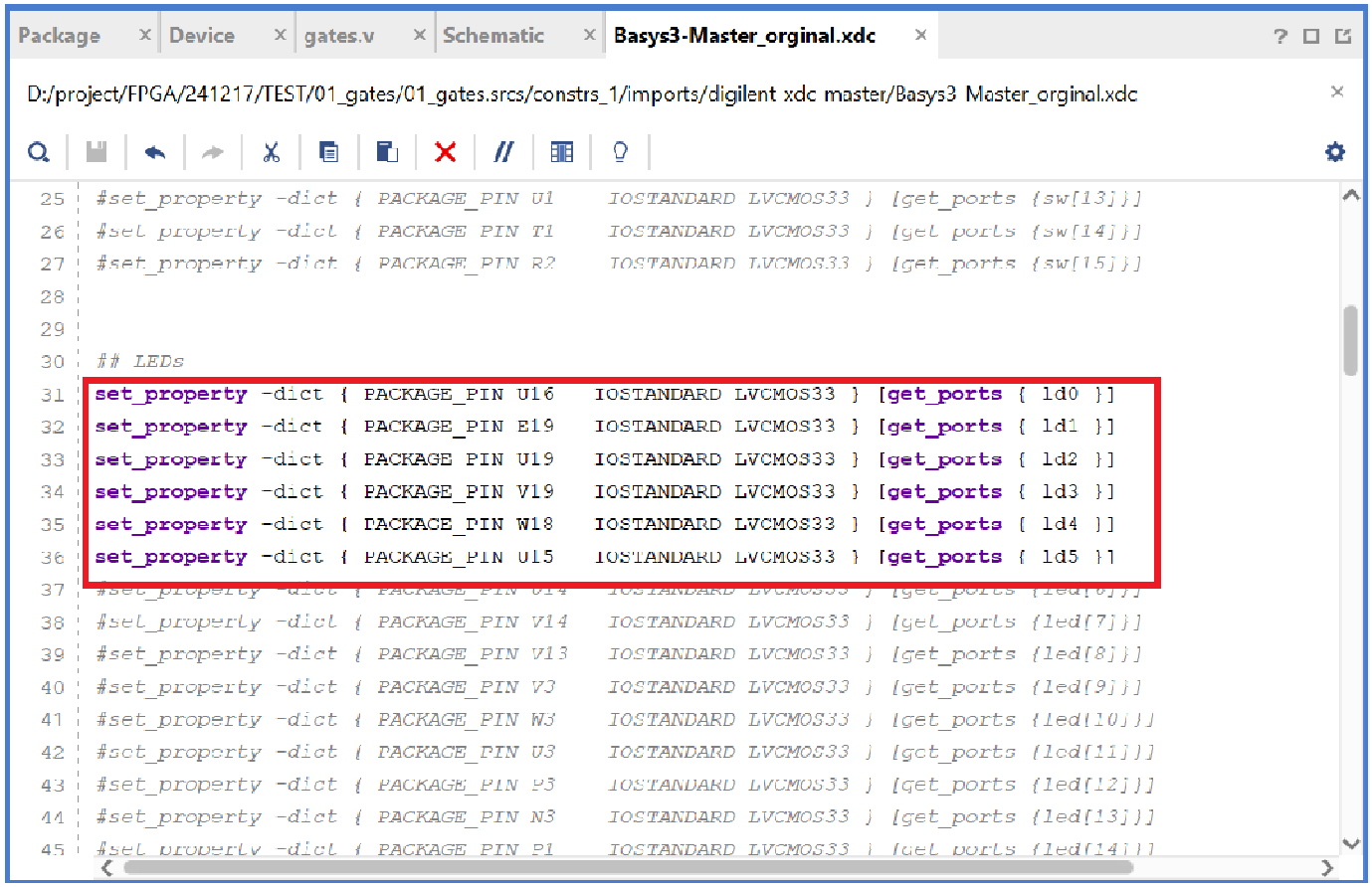

Sources
Netlist
Device Constraints
Package
Device
gates.v
Schematic
6 Cells
8 I/O Ports
Add Sources
Add or Create Constraints
Specify or create constraint files for physical and timing constraint to add to your project.
Specify constraint set:
constrs_1 (active)
Constraint File
Location
Basys3-Master_orginal.xdc
D:\project\FPGA\doc\BASYS3\digilent-xdc-master
Add Files
Create File
Copy constraints files into project
Back
Next
Finish
Cancel
Tcl Consol
Name
Scalar ports (8)
Slew

project 만들기

Constraints 파일을 source code에 맞게 mapping작업을 한다.



Constraints 파일 작성(pin constraints)

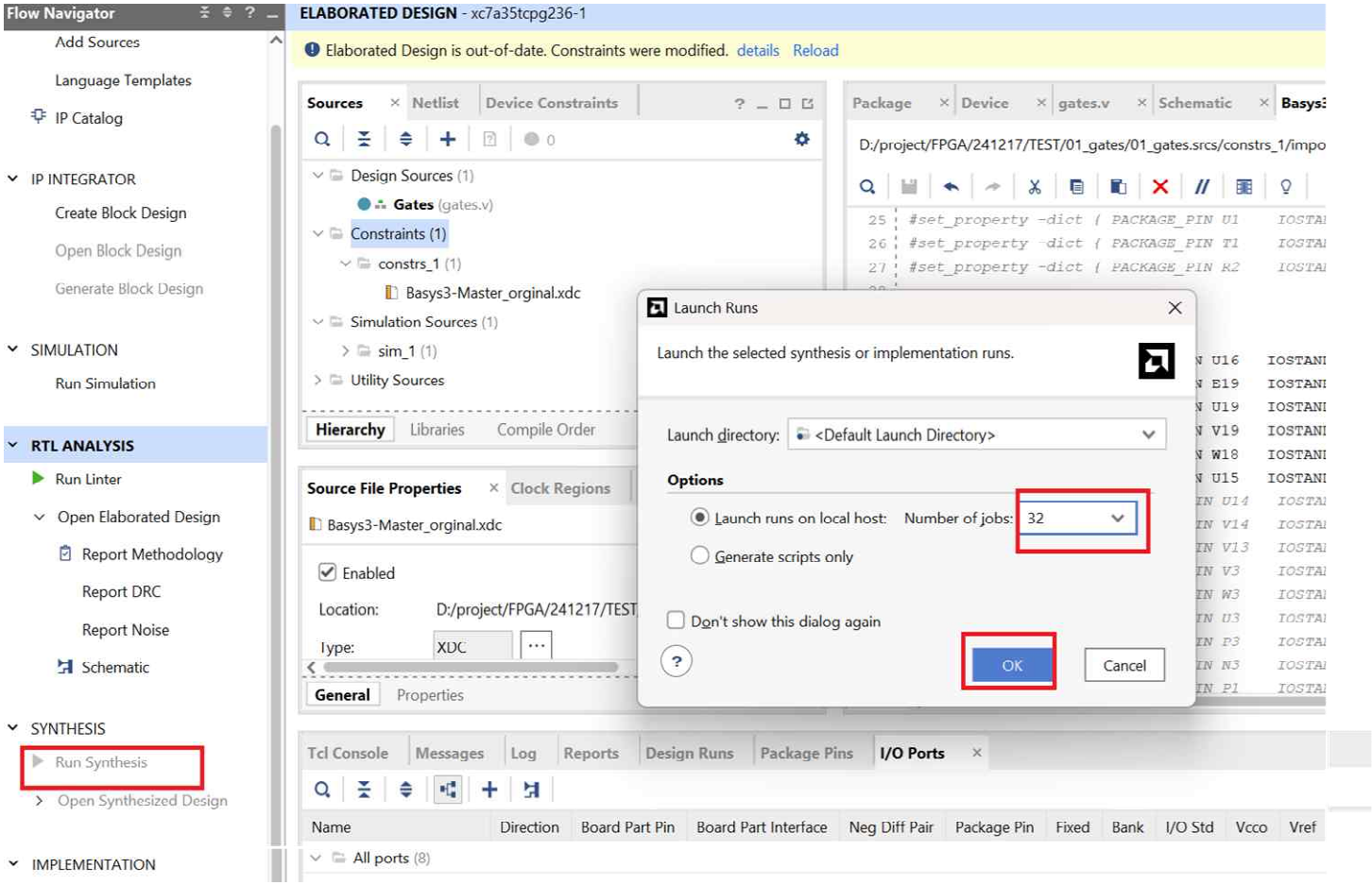


Constraints는 사전적 의미로
제한하다 라는 말로써
FPGA설계에 필요한 제한 조건을
만들기 위한 것이다. 예를 들어 가장 기본적인
Constraints에는 pin constraints가 있다.
Pin constraints란 FPGA Device에 HDL을
사용하여 설계한 하드웨어의 입출력 포트가
FPGA device의 어떤 pin을 사용할 것인지
지정 하는 것이다.

project 만들기 Synthesis

합성 한다. 아래 runs on local host의 개수를 최대로 지정 해준다. (PC사양 마다 지정 범위 다름)

Synthesis는 Verilog HDL code를 target device에서 제공하는 library에 맞춰서 H.W 회로로 변경



project 만들기 Synthesis

Vivado의 합성 엔진은 다음과 같은 순서로 논리식을 간소화.

•**1단계: RTL 분석 (Elaboration):** 개발자가 작성한 소스코드를 읽어 들여 문법을 검사하고, 하드웨어적인 기본 구조(RTL Netlist)로 변환. 이때는 코드에 적힌 그대로 논리 회로가 1:1 형태로 생성.
•**2단계: 논리 최적화 (Logic Optimization - 핵심 단계)**
인간이 손으로 **카르노맵(Karnaugh Map)**을 그리거나 **부울대수(Boolean Algebra)** 공식(드모르간 법칙, 분배법칙 등)을 써서 식을 줄이듯, Vivado 내부의 강력한 최적화 알고리즘(예: Espresso 알고리즘 계열이나 AIG 기반 최적화)이 작동.

- 중복된 논리 게이트 제거
- 도달할 수 없는 데드 코드(Dead Code) 및 무효한 신호 경로 제거
- 출력에 영향을 주지 않는 입력 조건을 무시하여 식을 최소화

•**3단계: 기술 매핑 (Technology Mapping):** 간소화가 완료된 순수 논리식을 자일링스(Xilinx) FPGA의 실제 하드웨어 자원인 **LUT(Look-Up Table)**, 플립플롭(Flip-Flop), 멀티플렉서 등으로 매핑하여 최종 합성 넷리스트(Synthesized Netlist)를 만들어 낸다.

project 만들기 Synthesis

Netlist

회로 내의 모든 소자(게이트, 플립플롭 등)와 이들을 연결하는 배선(net)을 기술한 리스트이다
Verilog로 설계한 회로가 실제 하드웨어에서 어떻게 구성될지를 나타낸 회로 연결망(네트워크 리스트

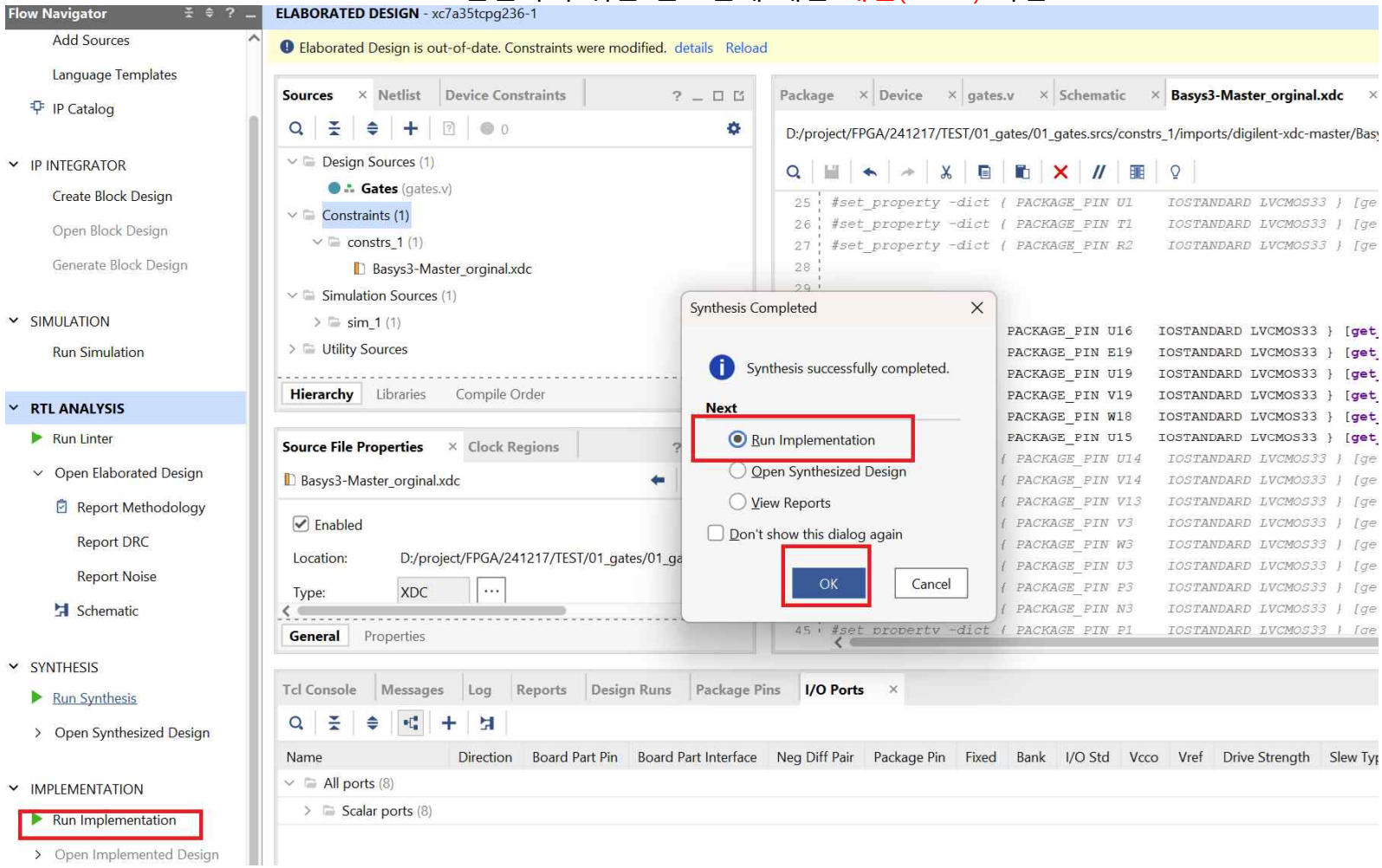
Netlist는 언제 생성되나?

Verilog 코드를 시뮬레이션이나 합성(Synthesis)을 위해 처리할 때 Netlist가 생성.
단계에 따라 종류가 달라짐.

단계	Netlist 종류	특징
전처리/시뮬레이션	RTL Netlist	원래 Verilog 구조 유지 (하이레벨)
합성(Synthesis) 후	Gate-level Netlist	실제 게이트들로 변환된 버전 (AND, OR, FF 등)
배치 후	Physical Netlist	실제 배선, 타이밍 정보 포함 (레이아웃 단계)

project 만들기 Implementation(회로 배치)

Implementation(회로 배치) : 설정된 입.출력 pin들에 맞춰 회로에서 사용된 자원들을 FPGA내부 구조에 맞도록 배치(place)하고 이들을 연결하기 위한 신호선에 대한 배선(route) 작업



project 만들기 Implementation(회로 배치)

① Opt Design (논리 및 물리적 최적화)
합성 단계에서 이미 논리식 간소화가 일어났지만, 구현 초기 단계에서 **물리적 배치를 고려한 추가 최적화**가 한 번 더 일어난다.
•사용할 필요가 없는 드라이버나 버퍼 게이트를 정리
•상수(0 또는 1)로 고정되어 움직이지 않는 무의미한 로직을 완전히 제거.

② Power Opt Design (전력 최적화 - 선택 사항)
칩의 전력 소모를 줄이기 위한 단계.
•동작하지 않고 쉬고 있는 플립플롭(Flip-Flop)이나 클럭 경로에 클럭 게이팅(Clock Gating)을 적용하여 전력 낭비를 차단.

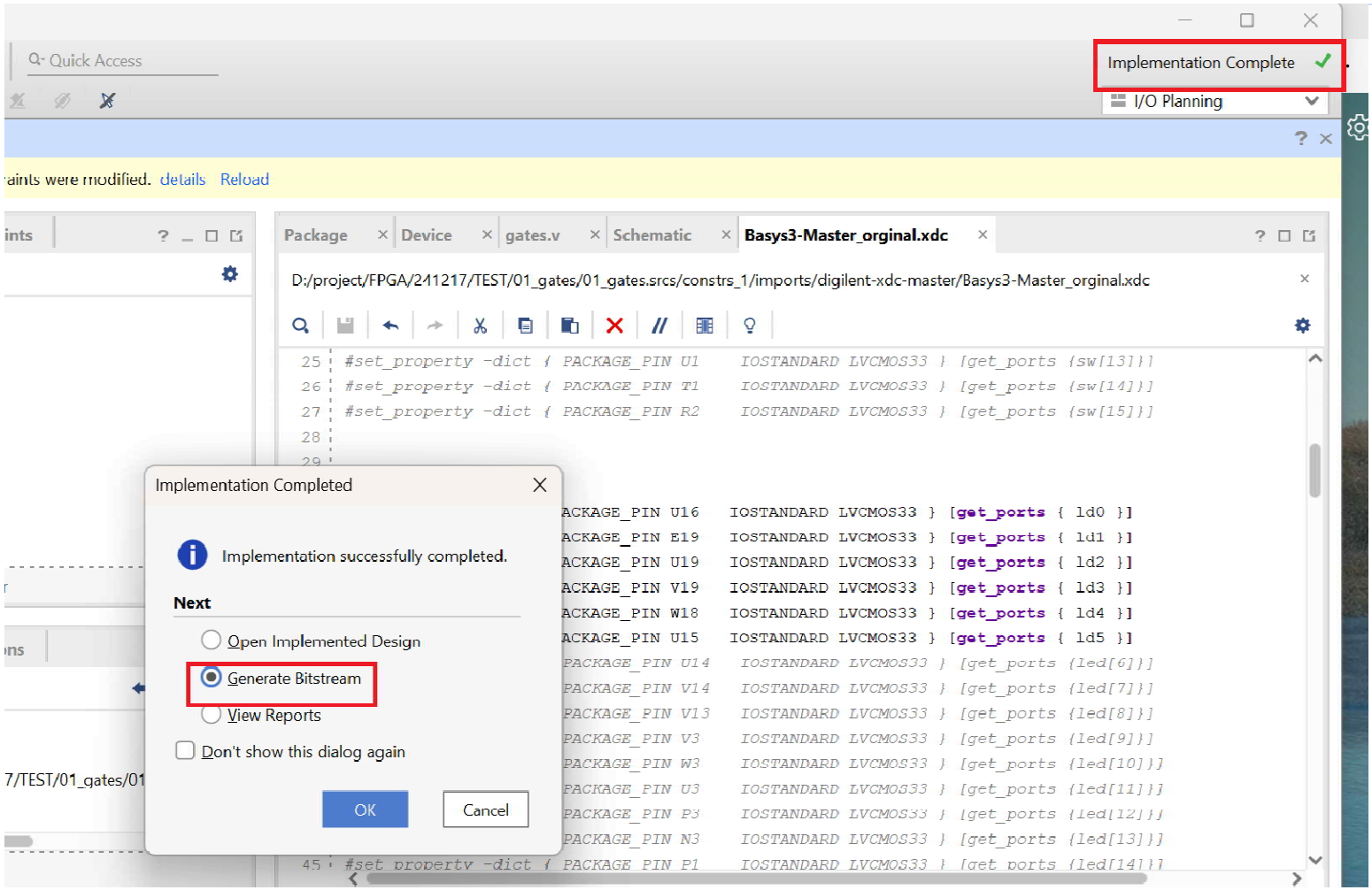
③ Place Design (배치 단계 - 매우 중요)
간소화된 논리 게이트(LUT, Flip-Flop 등)를 **FPGA 반도체 다이(Die) 위의 수많은 빈 슬롯 중 정확히 어느 물리적 좌표에 꽂을지 결정**하는 단계.
•서로 신호를 자주 주고받는 컴포넌트들은 신호 전달 지연 시간(Delay)을 줄이기 위해 최대한 가까운 위치에 뭉쳐서 배치.
•RAM이나 DSP처럼 칩 내부의 고정된 특수 자원의 위치를 고려하여 로직의 위치를 선정.

④ Phys Opt Design (물리적 배치 후 최적화)
배치가 완료된 상태에서, 타이밍(Timing) 성능을 만족하지 못할 것으로 예상되는 취약한 경로를 찾아내 보완.
•하나의 신호가 너무 많은 게이트로 뻗어나가 힘이 약해질 경우, 신호 복사본(Replication)을 만들어 분산.
•긴 물리적 경로 중간에 임시 보관소(레지스터)를 삽입하여 신호의 속도를 맞춤.

⑤ Route Design (배선 단계 - 매우 중요)
배치된 소자들 사이를 **FPGA 내부의 실제 물리적 구리선(Interconnect)으로 엮어 연결**하는 단계.
•가장 빠른 길(최단 경로)을 찾아 선을 연결.
•클럭(Clock) 신호처럼 칩 전체에 정확하고 동시에 퍼져야 하는 중요한 신호는 전용 글로벌 클럭 라인(Global Clock Network)에 우선 배정.
•신호선끼리 너무 가까워 발생하는 간섭(Crosstalk)이나 지연이 없도록 우회 경로를 뚫기도 한다.

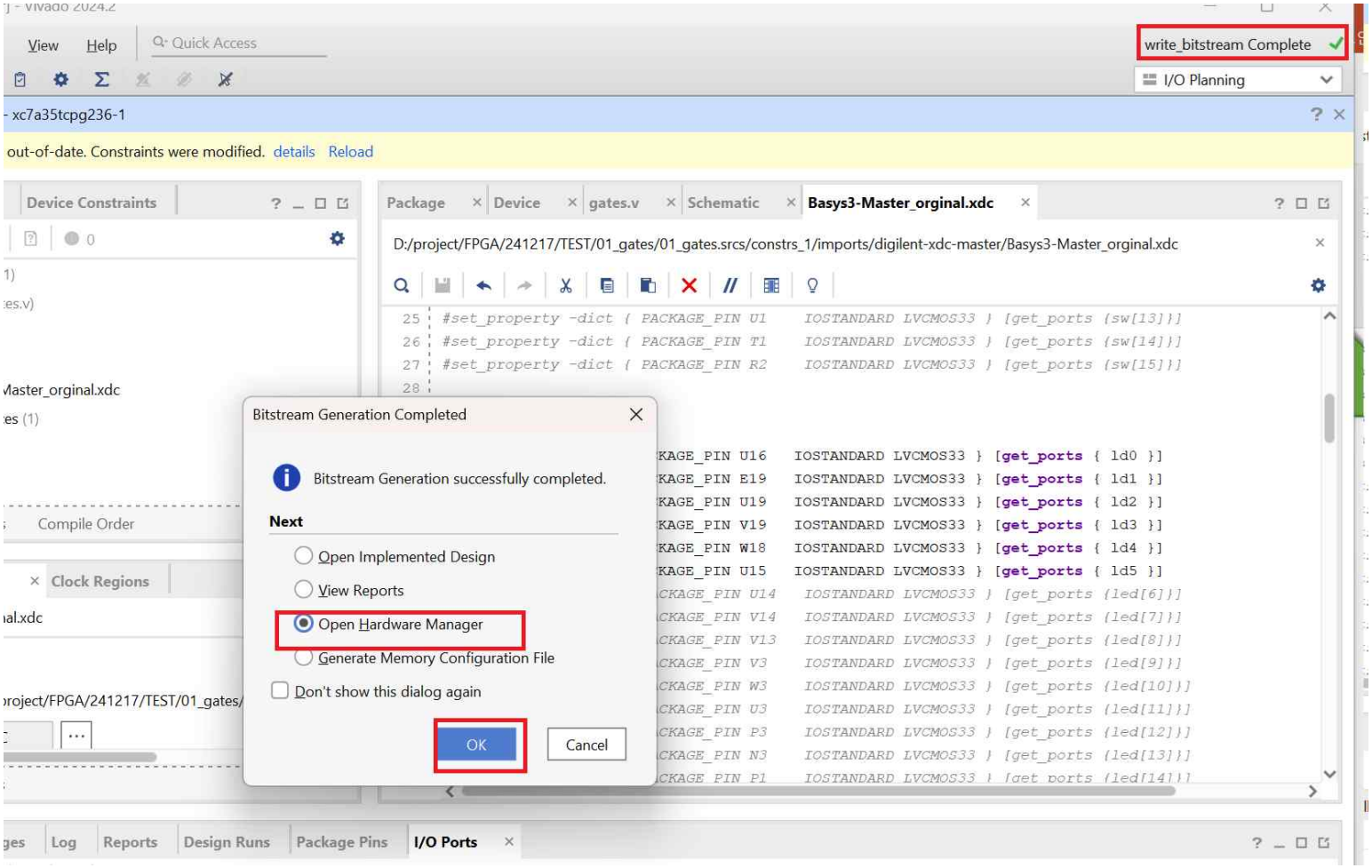
project 만들기 bitstream을 생성 한다

bitstream(FPGA board용 HW회로 파일)을 생성 한다. FPGA에 Download하기 위하여 FPGA가 인식할 수 있는 형태인 bit stream file을 생성 하는 작업



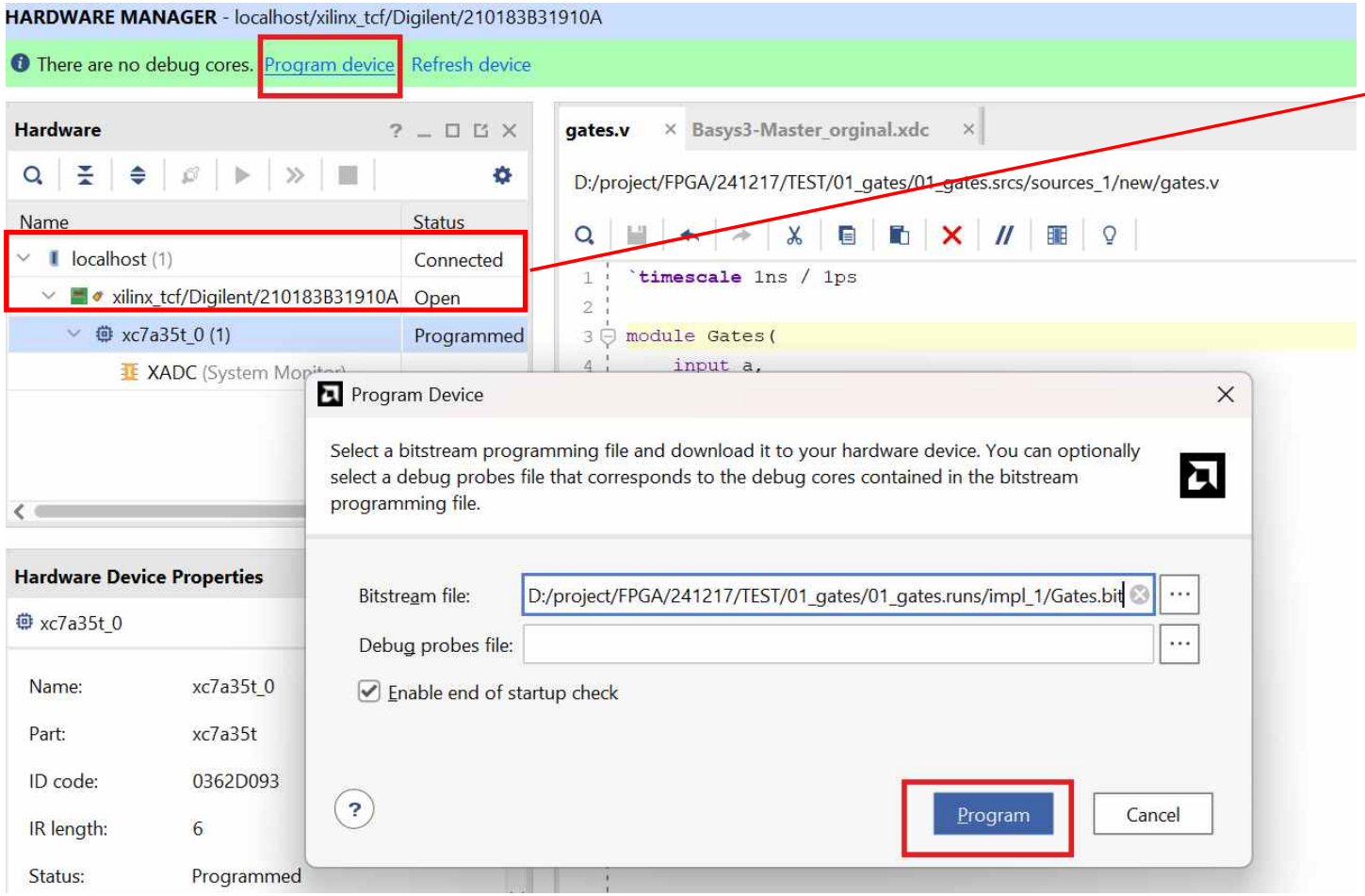
project 만들기 bitstream을 생성 한다

bitstream을 생성 한다



project 만들기

Targret (basys3에 bitstream을 다운로드 한다.) open target → Program device →



※ Target이 연결이 안될때
1. USBport를 바꿔본다
2. Xlinx>Vivado>2021.x>data >xicom>cable_drivers>nt64>digilent 프로그램을 설치후 PC를 reboot

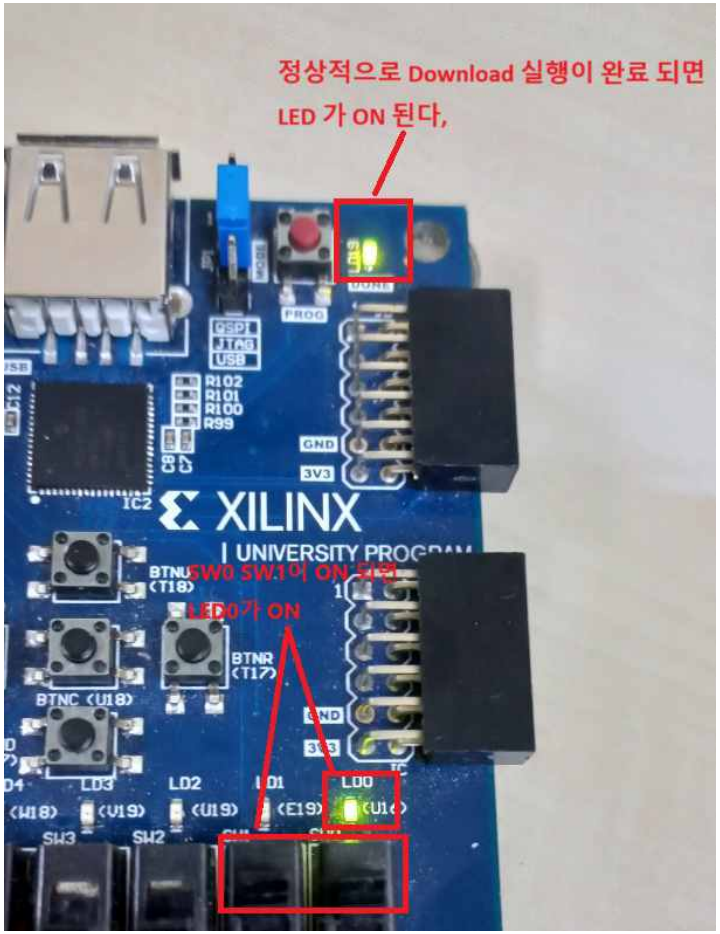
project 만들기

Targret (basys3에 bitstream을 다운로드 한다.) open target → Program device →

★ FLASH Memory에 programming 하기

project 만들기

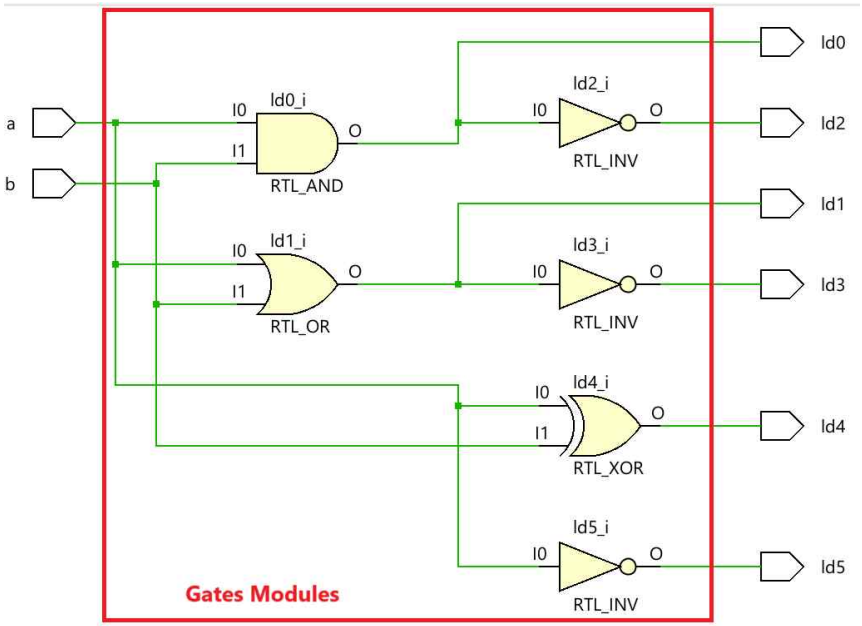
결과 확인



4. TEST BENCH 코드 만들기

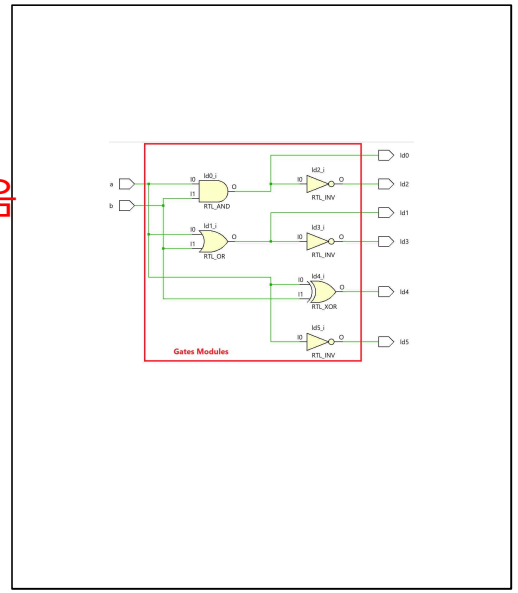
TEST BENCH 코드 만들기

Test bench는 가상의 검증 환경으로 검증 하고자 하는 회로에 적절한 입력 신호를 인가하고 그에 따른 출력을 보고 회로의 기능을 검증 한다. Test bench에는 입.출력 포트가 없다.



Module test bench

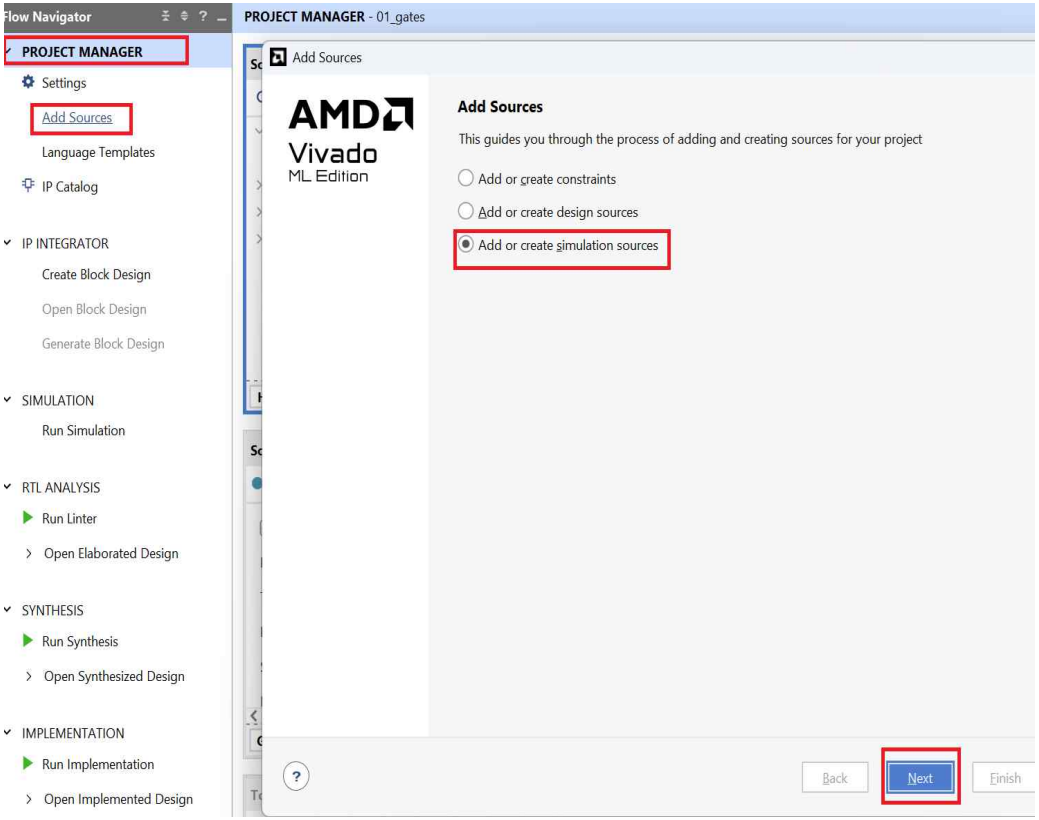
입력 없음

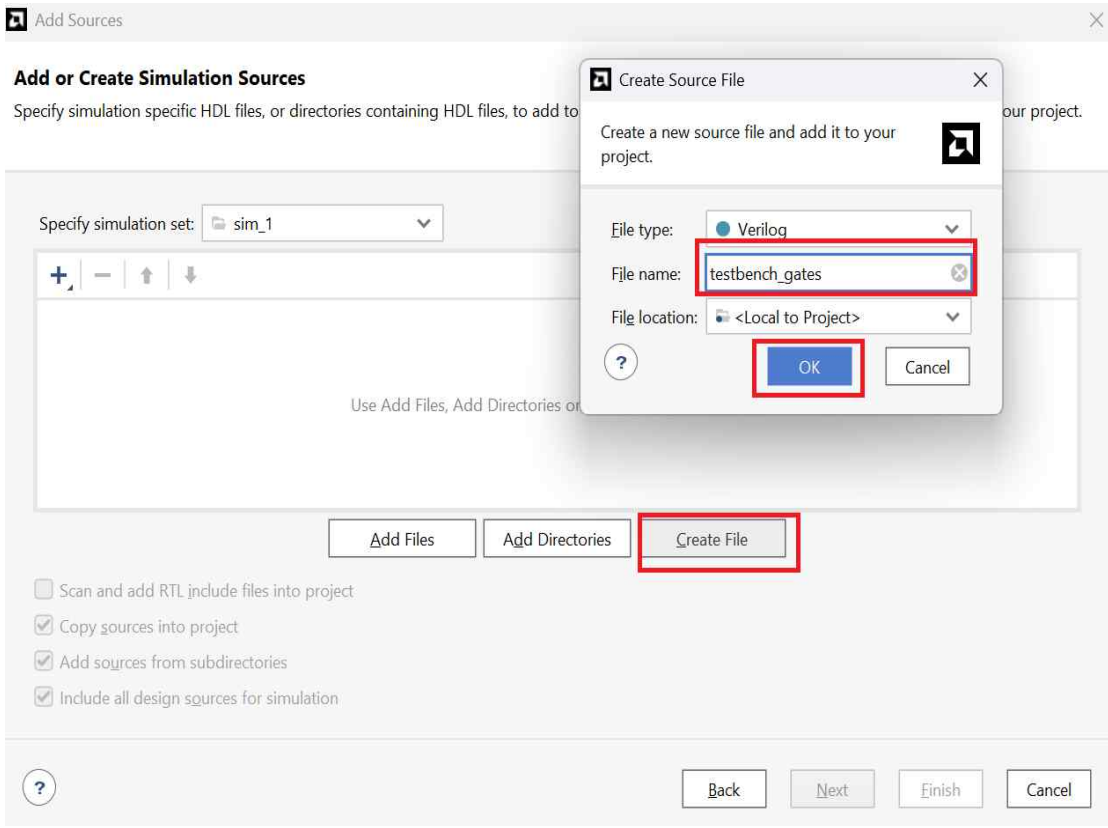


출력 없음

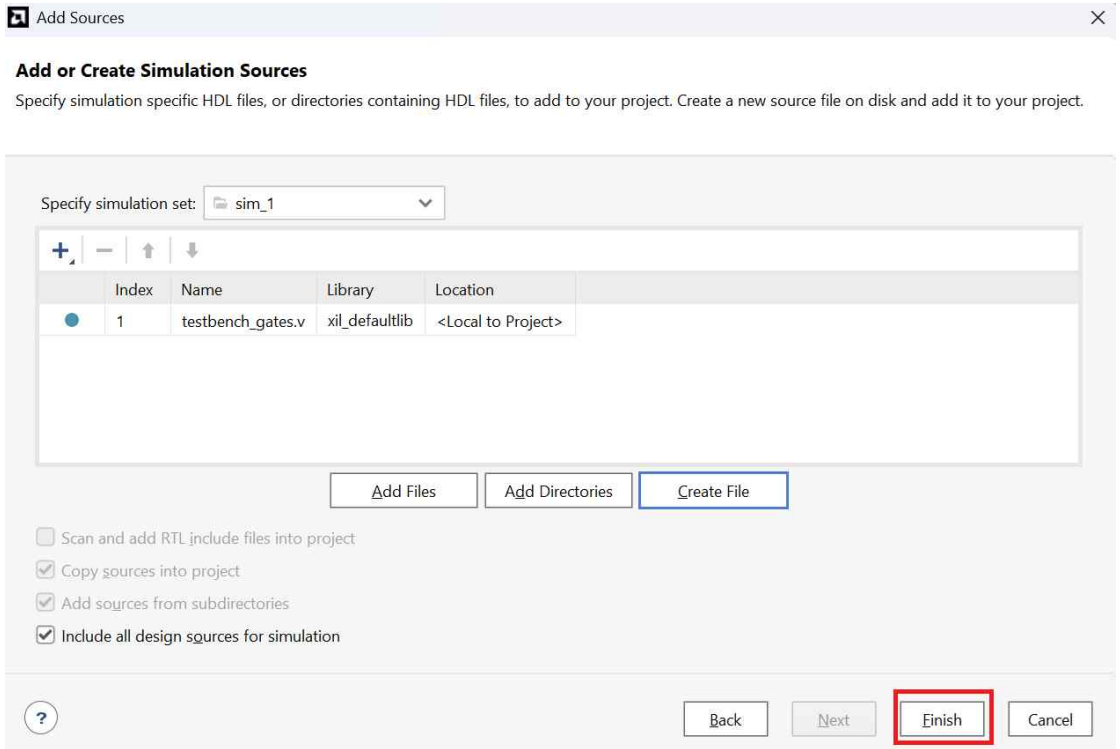
testbench_gates

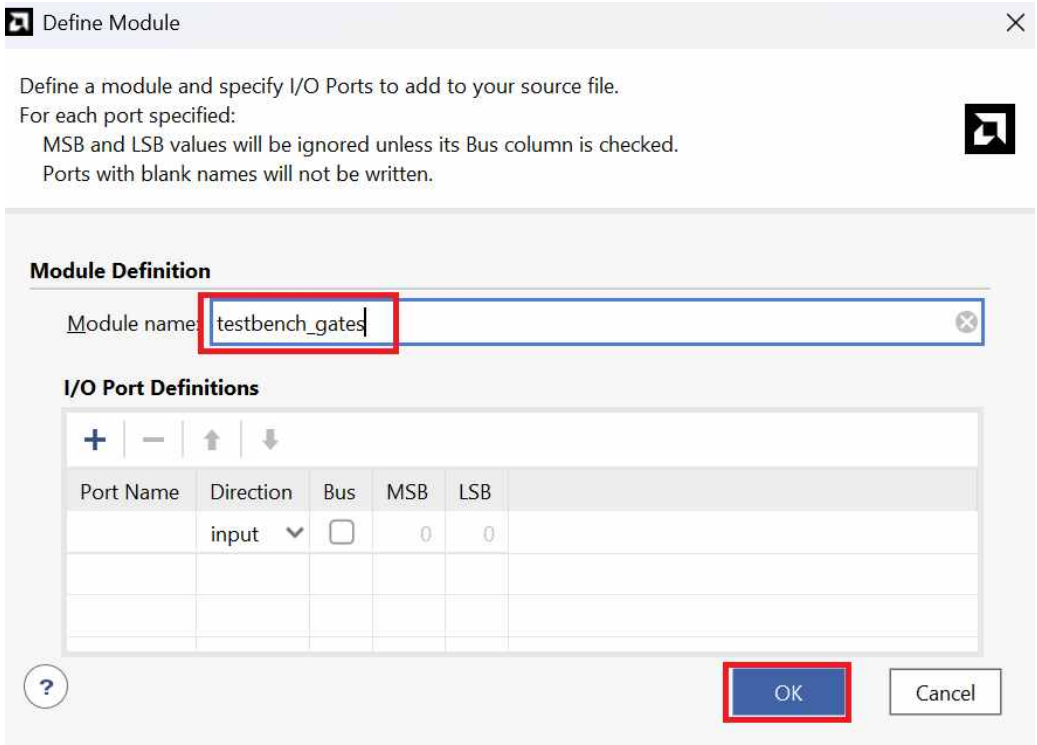
TEST BENCH 코드 만들기





TEST BENCH 코드 만들기





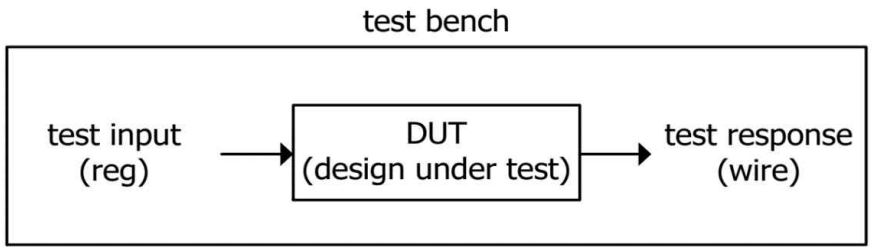
TEST BENCH 코드 만들기

■ DUT(Design Under Test)

- 실제로 구현 되어 사용할 로직, 검증하기 위한 HW 모듈
- 합성 가능한 구문으로 작성 (assign 또는 always 사용)

■ Test Bench

- DUT를 테스트하기 위한 Verilog 모듈
- 가상의 검증 환경으로, 검증하고자 하는 디자인에 적절한 입력신호(stimulus)를 인가하고 그에 따른 출력을 확인하여 설계한 회로의 기능을 검증하는 모듈이다.
- 합성이 가능하지 않은 구문도 사용 가능(initial, wait 등)
- test bench 모듈은 입.출력이 없다
- test input: 검증 하고자 하는 모듈의 입력신호를 모두 reg로 선언 한다.
- test output: 모든 출력 신호를 wire로 선언 한다
- test하고자 하는 모듈을 인스턴스화 하고 선언한 신호를 해당하는 포트에 연결 한다.



TEST BENCH 코드 만들기(initial 구문)

■ Test Bench

- Initial 구문
  . simulation이 실행 되면서 한번만 실행 되는 구문
  . 할당문들은 보통 blocking 할당문을 사용
  . 문장들은 나열된 순서대로 실행
  . 합성 되지 않는 문법

- Timescale
  `timescale 1ns/10ps

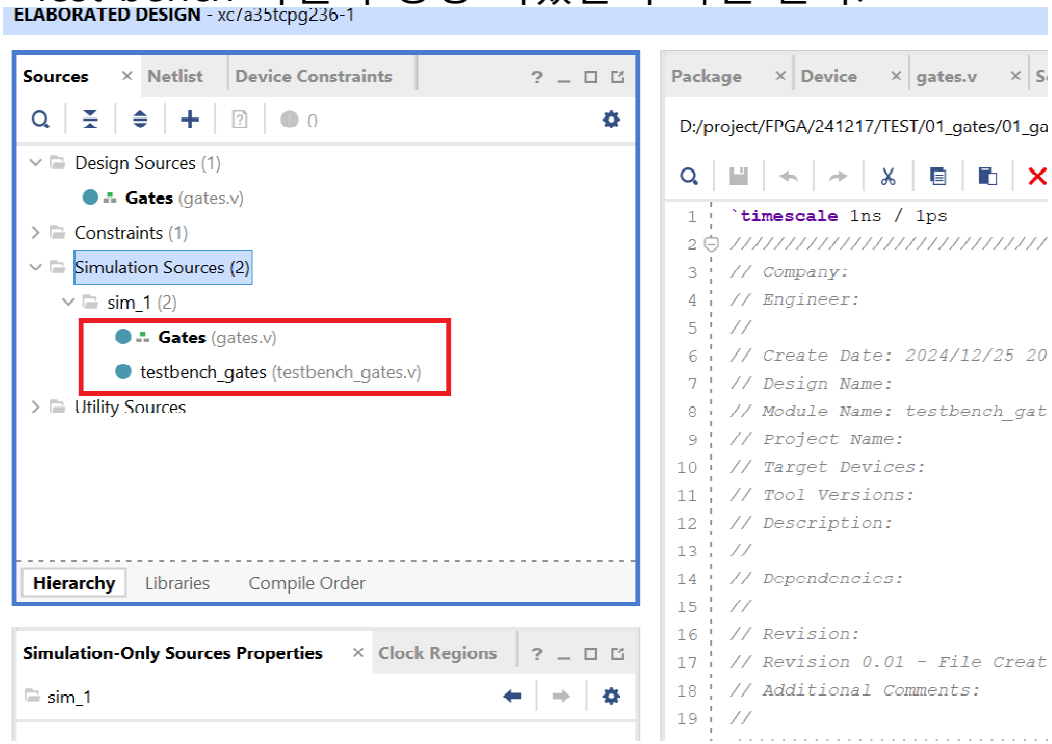
정밀도 (Time Precision): 시뮬레이션 시 소수점 이하 어디까지 계산할지를 결정. 10ps라면 0.01ns 단위까지

`timescale (time unit) / (time precision) `timescale 1ns/10ps	
time unit	시간 단위가 1 ns 임 (예, #10 은 10ns를 의미함)
time precision	Time precision이 10ps이므로 지연값은 0.01ns의 정밀도를 갖는다. 즉 소수점이하 2자리의 정밀도로 반올림 한다.

예1) `timescale 1ns / 1ps
  -> time_unit이 1ns로 지정 되었으므로 이후의 시간 값들은 1ns의 배수가 된다.
  time_precision이 1ps이므로 지연값은 0.001ns의 정밀도를 갖는다. 즉 소수점 이하 3자리의 정밀도로 반올림 한다.
예2) `timescale 10us / 100ns
  -> time_unit이 10us로 지정 되었으므로 이후의 시간 값들은 10us의 배수가 된다.
  time_precision이 100ns이므로 지연값은 0.1us의 정밀도를 갖는다. 즉 소수점 이하 1자리의 정밀도로 반올림 한다.

TEST BENCH 코드 만들기

Test bench 파일이 생성 되었는지 확인 한다.



Test bench code를 작성 한다

```
`timescale 1ns / 1ps

module testbench_gates();
    reg a;
    reg  b;
    wire ld0;
    wire ld1;
    wire ld2;
    wire ld3;
    wire ld4;
    wire ld5;

Gates dut(
    .a(a),
    .b(b),
    .ld0(ld0),
    .ld1(ld1),
    .ld2(ld2),
    .ld3(ld3),
    .ld4(ld4),
    .ld5(ld5)
    );

    initial begin
        #00 a = 1'b0; b = 1'b0;
        #10 a = 1'b0; b = 1'b1;
        #10 a = 1'b1; b = 1'b0;
        #10 a = 1'b1; b = 1'b1;
        #10 $finish;
    end

endmodule
```

testbench는 입력이 존재 하지 않는다

1bit 저장공간

모듈이름

인스턴스 이름

시뮬레이션용 keyword

10ns만큼 delay

10ns 이후에 종료

TEST BENCH 코드 만들기

a b ld0 ld1 ld2 ld3 ld4 ld5 의 module Gates에 있는 input outpu과 구분 하기 위하여 아래와 같이 이름을 달리 주어 test bench code를 작성(좌측 하단) simulation결과 (우측 하단)
아래는 모듈을 연결(Instantiation) 할때는 named instantiation(연결)을 사용한다.

```
`timescale 1ns / 1ps

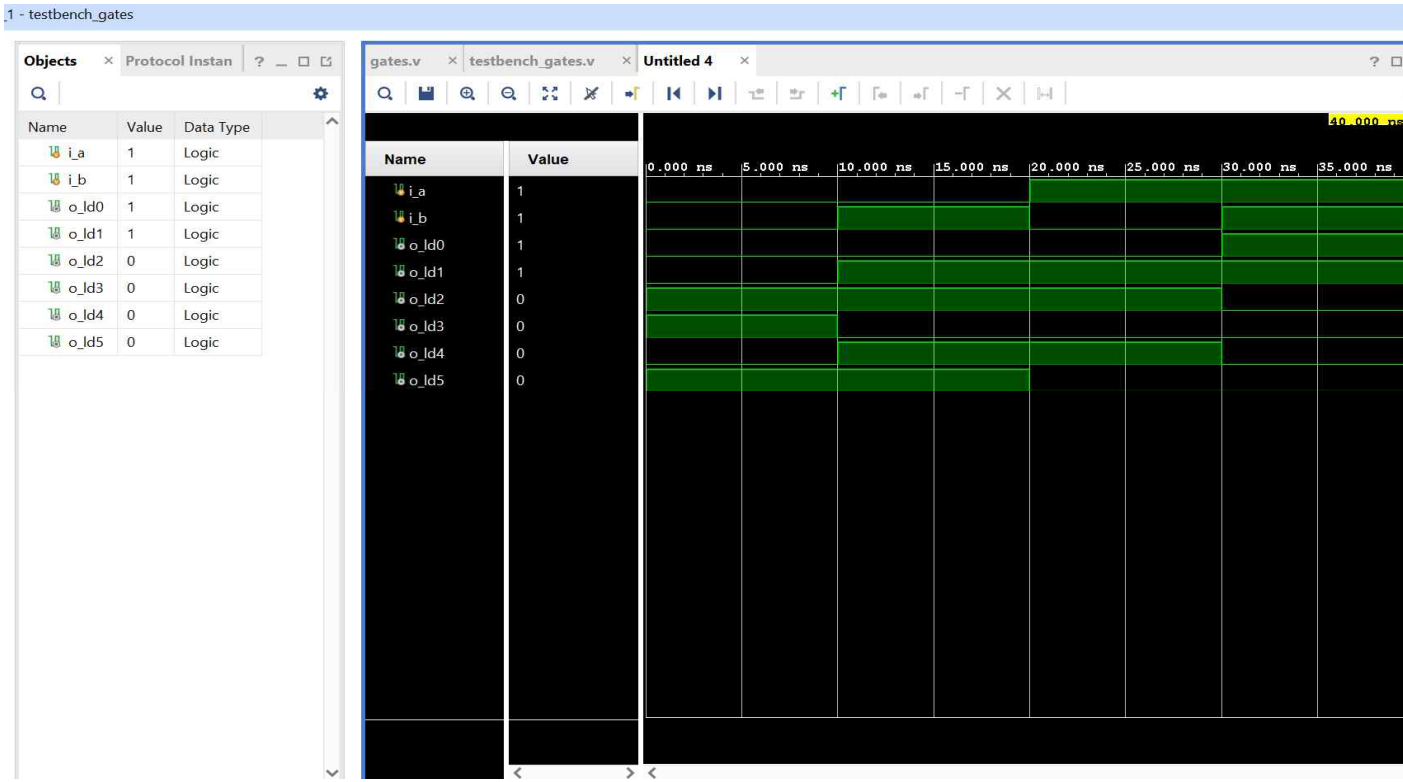
module testbench_gates();

    reg i_a;
    reg  i_b;
    wire o_ld0;
    wire o_ld1;
    wire o_ld2;
    wire o_ld3;
    wire o_ld4;
    wire o_ld5;

Gates dut(
    .a(i_a),
    .b(i_b),
    .ld0(o_ld0),
    .ld1(o_ld1),
    .ld2(o_ld2),
    .ld3(o_ld3),
    .ld4(o_ld4),
    .ld5(o_ld5)
    );

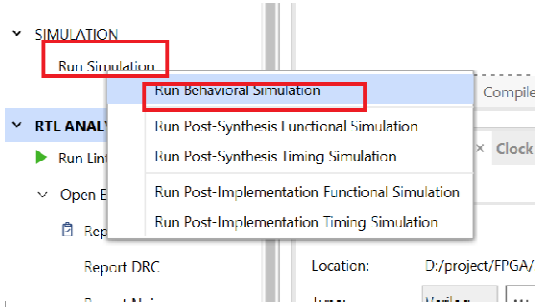
   initial begin
        #00 i_a = 1'b0; i_b = 1'b0;
        #10 i_a = 1'b0; i_b = 1'b1;
        #10 i_a = 1'b1; i_b = 1'b0;
        #10 i_a = 1'b1; i_b = 1'b1;
        #10 $finish;
   end

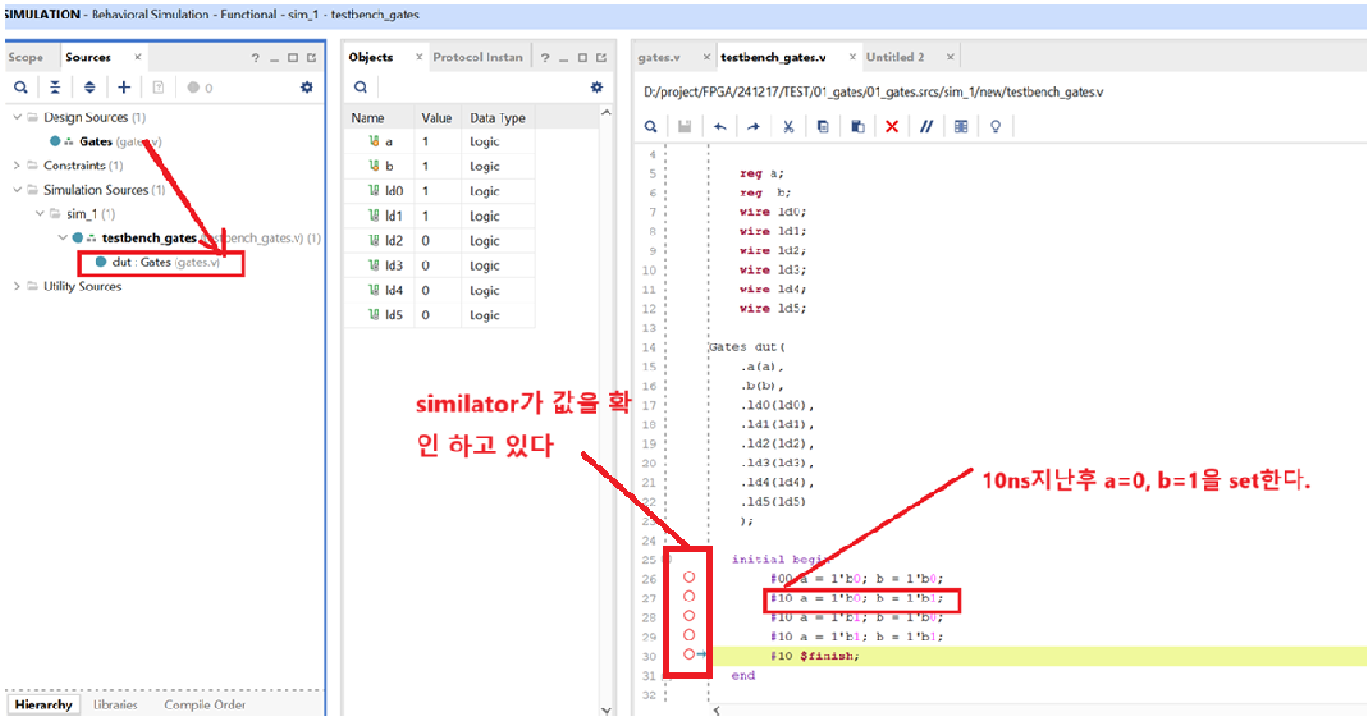
endmodule
```



TEST BENCH 코드 만들기

Simulation을 실행 한다





TEST BENCH 코드 만들기

Simulation 결과 보기



TEST BENCH 코드 만들기

Simulation 결과 보기

